https://pubsonline.informs.org/journal/opre

Submitted to *Operations Research*

OPERATIONS RESEARCH

Vol. 00, No. 0, Xxxxx 0000, pp. 000–000

ISSN 0030-364X, EISSN 1526-5463

Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints

(Authors' names are not included for peer review)

Abstract. Large language models (LLMs) now serve millions of users daily, with some providers incurring compute costs exceeding $700,000 per day. Each request requires running inference, the process of generating a response token by token, and the scheduler at the server determines how many concurrent requests a given GPU can handle. We study online scheduling for LLM inference under an unusual memory constraint: each request's Key-Value (KV) cache, the intermediate attention state reused across output tokens, grows with every new token, so a request's memory consumption is nonstationary and uncertain at arrival. When memory fills up, the scheduler must evict in-progress requests and discard their prior computation. Even when capacity is theoretically sufficient for stability, a naive policy can trigger cascading evictions that collapse the system; the risk is especially acute when output lengths are unknown at arrival, because the scheduler cannot plan ahead. We formulate LLM inference as a multi-stage online scheduling problem and analyze a fluid approximation that characterizes the ideal-fluid equilibrium, the corresponding memory requirement, and the resulting stability benchmark. Guided by this benchmark, we design the WAIT (Waiting for Accumulated Inference Threshold) algorithm, a threshold-based admission rule that keeps the in-system state near the ideal-fluid equilibrium and prevents eviction cascades. When output lengths are known, WAIT achieves asymptotically optimal effective throughput, the completion rate net of eviction. For unknown output lengths, we extend the design to Nested WAIT, which uses nested thresholds across decode segments so that short prompts exit early while long prompts advance to later segments, with no output-length prediction required. A safety buffer that is logarithmic in the failure probability prevents memory overflow with high probability. Both algorithms are asymptotically optimal in the long-horizon regime. On Llama-2-7B with an NVIDIA A100 GPU, WAIT and Nested WAIT enlarge the realized stability region, raise effective throughput, and reduce mean end-to-end latency relative to vLLM and Sarathi, two widely used open-source LLM inference systems, across a wide range of arrival rates.

Key words: Large Language Model, Key-value cache, Memory Constraint, Online scheduling

1. Introduction

Large Language Models (LLMs) now dominate natural language processing (NLP) (Devlin et al. 2019, Brown et al. 2020, Kaplan et al. 2020, Ouyang et al. 2022, Wei et al. 2022a,b, Touvron et al. 2023, OpenAI 2024), powering chatbots (Anthropic 2023, Bai et al. 2023, OpenAI 2024, DeepSeek-AI 2025), search engines (Google 2023, Microsoft 2023), and programming assistants (GitHub 2022, Anthropic 2025). These models generate text through *inference*, which imposes

significant computational demands on memory and processing resources (García-Martín et al. 2019). Systems like ChatGPT incur daily inference costs exceeding $700,000 and contribute to substantial carbon emissions (Patel 2023, Patterson et al. 2021). Efficient inference scheduling can reduce these operational costs and energy consumption (Strubell et al. 2019, Desislavov et al. 2021) by balancing system throughput and response latency (Wu et al. 2022).

Figure 1 illustrates how a query is processed during LLM inference. The user submits a prompt (e.g., "Is apple a fruit?"), which then passes through two computational phases:

- *Prefill phase (stage 0).* Upon receiving the prompt, the model first tokenizes the input into a sequence of discrete units (e.g., "Is", "apple", "a", "fruit", "?"). These tokens are then embedded and simultaneously processed in a single forward pass to compute the *Key-Value (KV) cache*, which stores intermediate representations (i.e., attention keys and values) for each token. These precomputed values enable efficient reuse during subsequent decoding steps. This phase corresponds to Stage 0 in Figure 1, where all prompt tokens are embedded and their KV representations are added to the cache.
- *Decode phase (stages 1 to l').* After the prefill phase, the model enters the decode phase, where it generates the output one token at a time. At each stage, the model queries the existing KV cache to compute the next token, appends the new token to the output sequence, and updates the KV cache with its key-value pair. For instance, the model might first generate "Yes" (Stage 1), then "it" (Stage 2), followed by "is" (Stage 3), and finally the "End of Sequence" (EOS) token (Stage 4). This progression is depicted in the lower part of Figure 1. Each token generation step involves both reading from and writing to the KV cache, so the KV cache grows linearly with the length of the generated sequence.

Figure 1 also highlights key performance metrics in LLM inference:

- *Time-to-first-token* (TTFT) measures the latency from user input to the first generated token.
- *Latency* refers to the total time required to complete the generation of all output tokens.
- *Throughput* captures the average number of tokens generated per unit time.

This inference structure, particularly the growing memory requirements and sequential decoding pattern, introduces fundamental constraints on scheduling and batching. Efficient scheduling must account for both prompt heterogeneity and KV cache dynamics to balance latency and resource use.

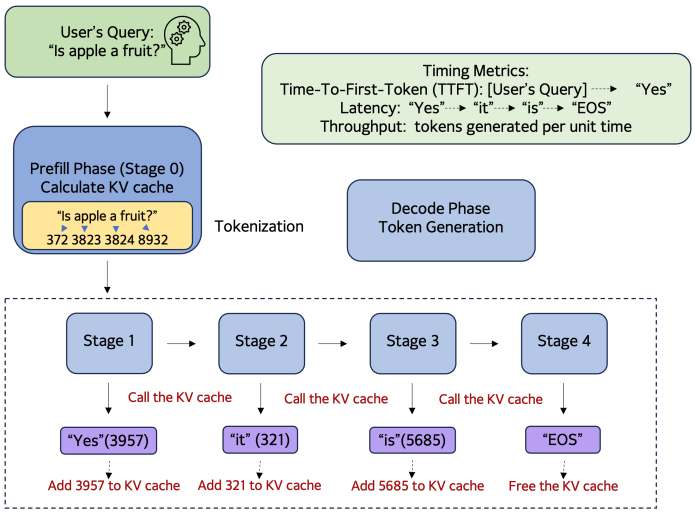

Figure 1 An example of LLM inference.

Scheduling LLM inference tasks involves grouping prompts into *batches* processed concurrently on a GPU. Batching improves throughput: processing multiple prompts together amortizes the fixed overhead of each iteration and better uses GPU parallelism. However, this benefit comes with a fundamental tension: larger batches consume more memory through their combined KV caches. The KV cache improves efficiency, as it prevents the model from recalculating attention history for each new token; without it, computational cost would scale quadratically with sequence length. But the KV cache also grows dynamically during decode, making memory consumption unpredictable. This creates a core trade-off: *larger batches increase throughput but consume more memory, risking capacity overflow*.

Memory overflow can occur in several practical scenarios: (i) *bursty arrivals*, where sudden traffic spikes temporarily exceed capacity; (ii) *unknown output lengths*, where prompts generate longer responses than anticipated; and (iii) *multi-tenant environments*, where multiple users compete for GPU memory. The unknown output length problem is particularly challenging: since the scheduler cannot predict how much memory each prompt will eventually consume, it cannot reliably plan batch composition or admission decisions. Traditional scheduling methods such as Shortest Job First (SJF) assume fixed job sizes and known processing times, making them unsuitable when memory demands grow unpredictably (Tay et al. 2022, Kang et al. 2024, Hooper et al. 2025).

When the total KV cache exceeds GPU memory capacity, the system must evict some in-progress prompts. Two approaches exist: (i) *swapping* to CPU or SSD, which incurs I/O overhead (Sheng et al. 2023, Aminabadi et al. 2022); and (ii) *recomputation*, which discards the KV cache and restarts

the prompt from prefill (Kwon et al. 2023). We focus on recomputation because it is the default in modern production systems such as vLLM (vLLM Team 2025), and because our goal is to *minimize eviction* through scheduling rather than to optimize the eviction mechanism itself (Li et al. 2025a). Eviction creates a vicious cycle: it frees memory temporarily, but restarted prompts consume the memory again and can trigger further evictions. Users experience this as error messages (e.g., "Something went wrong") or incomplete responses under heavy load.

Memory sufficiency alone does not guarantee stability. The key to approaching maximal effective throughput is *load balance*: arrivals, completions, and memory growth must remain aligned. Supporting such an operating point requires enough memory for the corresponding mix of active prompts, summarized later by the memory requirement M^*. Even when total capacity C exceeds this requirement, naive policies can push the stochastic system away from the corresponding allocation of active prompts across decode stages. For instance, first-come-first-served (FCFS) can trigger cascading evictions that reduce effective throughput, the completion rate net of evictions, by as much as 20% when capacity is theoretically sufficient (Example 2). A slight imbalance causes one prompt type to overflow, triggering evictions; the freed memory then fills with another type, which itself overflows, creating a vicious cycle. Thus, a system that *should* be stable becomes unstable under poor scheduling.

Preventing eviction requires *approaching load balance through controlled admission*. The key scheduling decision is the composition of each inference iteration: which prompt types and decode stages enter the next batch. This composition determines how much fixed overhead is shared in the current iteration, how much KV cache is carried forward, and whether future iterations approach the overflow boundary. Our algorithms use thresholds to regulate this composition. A prompt advances only when the corresponding threshold has enough room, keeping the in-service population close to the equilibrium allocation across decode stages. Operating near this allocation *enlarges the realized stability region*, because eviction cascades no longer destabilize a system whose capacity is theoretically sufficient, and *reduces mean end-to-end latency most clearly near capacity and under overload*. In underloaded regimes, our algorithms and the baselines are often close; as load increases, baseline latency grows sharply and completion rates saturate earlier, while our algorithms keep the system closer to the offered load. Section 6 documents these effects.

Recent system-level optimizations for LLM inference (Yu et al. 2022, Kwon et al. 2023, Agrawal et al. 2023, Pope et al. 2023, DeepSeek-AI 2024, Patel et al. 2024, Zhong et al. 2024) focus

on engineering solutions but lack a rigorous mathematical foundation. In practice, output length predictions are imprecise or costly (Fu et al. 2025), and algorithms designed for known output lengths can degenerate significantly when this assumption fails (see Proposition 5).

We develop a fluid approximation that formalizes this operating point as an ideal-fluid equilibrium, where prompt arrivals and completions balance, and characterizes the memory requirement M^* needed to support it. For a fixed capacity C, the condition $M^* \le C$ defines the ideal-fluid stability benchmark. This benchmark guides the threshold design: when the stochastic system operates near the corresponding equilibrium allocation across decode stages, it sustains high effective throughput while avoiding eviction. The analysis shows that memory is a resource to exploit rather than merely a constraint, because larger batches process more prompts per iteration and increase GPU utilization.

Achieving this equilibrium in the stochastic system is not automatic, because each admission decision changes both the current batch and the future KV-cache state. We therefore use the structure of the fluid equilibrium point to construct threshold policies. With known output lengths, WAIT sets admission thresholds from the reference populations across decode stages: a prompt advances only when the corresponding decode stage has room, so admission restores the load-balanced allocation rather than creating downstream memory pressure. With unknown output lengths, Nested WAIT applies the same rule through nested decode segments: short prompts complete and release memory early, while prompts that remain active advance to later segments without output-length prediction. The threshold structure reduces the original memory-coupled control problem to lower-dimensional queueing dynamics while preserving the batching benefits needed for high effective throughput.

1.1. Summary of Contributions

Our main contributions are as follows.

- *Memory-constrained scheduling model.* We develop a multi-stage online scheduling model that captures the dynamic growth of KV cache memory during LLM inference, where exceeding capacity triggers costly prompt evictions (Section 2). With fixed external arrivals, a stable policy completes work at the arrival rate, so the design goal is twofold: to approach the ideal-fluid stability benchmark, and to operate near the equilibrium allocation across decode stages supported by the memory requirement M^*. A fluid approximation from queueing theory lets us characterize both (Section 3).

- *Eviction-prevention scheduling algorithms.* We develop threshold-based algorithms that prevent memory overflow and minimize eviction. The WAIT algorithm (Section 4) handles known output lengths; the Nested WAIT algorithm (Section 5) extends to unknown output lengths through on-the-fly type classification. A threshold at each decode-stage boundary performs a *dimensionality reduction*: it decouples prompt types and replaces the continuously growing KV cache dynamics with a discrete-time per-type recursion. Both algorithms are asymptotically optimal.
- *Experimental validation.* We evaluate our algorithms on synthetic and real-world workloads using Vidur (Agrawal et al. 2024a), a widely used LLM-serving simulator, configured for Llama-2-7B on a single A100 GPU (Section 6). We compare against vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023), two widely used open-source LLM inference servers that represent, respectively, memory-reactive preemptive recomputation and chunked-prefill scheduling. We also validate the iteration-time model and implementation behavior using real-GPU measurements on L20 and A100 hardware (Section 2 and Appendix H.2). Consistent with this load-balancing mechanism, WAIT and Nested WAIT *enlarge the realized stability region* relative to both baselines and reduce mean end-to-end latency most visibly near capacity and under overload.

Our batching theory also extends naturally to Prefill-Decode Disaggregated (PD) systems such as DistServe (Zhong et al. 2024) and Splitwise (Patel et al. 2024), in which the decode server handles decode-only workloads. In this setting, the affine iteration-time model in Section 2 is a close approximation after calibration, because prefill attention effects are separated from decode service. The same threshold construction applies on the decode side, and Appendix H.4 reports numerical results on representative PD workloads.

1.2. Other Related Work

Online Scheduling Problem and Queueing System. Classical online scheduling problems focus on optimally assigning jobs that arrive sequentially, each with varying completion times. In settings with stochastic arrivals, foundational studies such as Devanur and Hayes (2009), Vee et al. (2010), Cole and Roughgarden (2014), Balkanski et al. (2016), Lattanzi et al. (2020) have developed algorithms that learn arrival patterns or distributions to refine scheduling strategies over time. Beyond individual job processing, works like Im and Moseley (2013), Lucier et al. (2013), Liu and Lu (2015), Li et al. (2020) have explored simultaneous batching and scheduling, grouping similar jobs to enhance efficiency. Within queueing theory, fluid models serve as first-order approximations of stochastic systems, enabling near-optimal control strategies Mandelbaum et al. (1998),

Maglaras (2000), Bäuerle (2002), Liu and Whitt (2011), while asymptotic analysis characterizes long-horizon behavior. However, these traditional approaches do not account for key features of LLM inference that complicate asymptotic analysis: (i) iteration time varies with batch composition and current KV cache sizes, precluding fixed service time assumptions; (ii) memory consumption grows dynamically during decode as new tokens are generated; (iii) out-of-memory (OOM) events trigger prompt eviction, creating feedback loops that classical stability and limit theorems cannot accommodate, as those theorems assume jobs complete once started, whereas eviction invalidates standard drift arguments.

LLM Inference. Theoretical frameworks for analyzing Large Language Model (LLM) inference remain scarce despite the scale of production deployment. Unlike classical scheduling, LLM inference combines stochastic arrivals, dynamic memory constraints, and multi-phase prefill/decode processing. System-level works such as Splitwise (Patel 2023) and DistServe (Zhong et al. 2024) split inference into separate prefill and decode pipelines, while scheduling engines such as Orca (Yu et al. 2022) and Sarathi (Agrawal et al. 2023, 2024b) improve throughput through batching and admission control. Other system-level optimizations include multi-head latent attention with low-rank KV compression (DeepSeek-AI 2024) and Kendall's-Tau prioritization by predicted output length (Fu et al. 2025). These works advance system-level efficiency through implementation and engineering. We complement them from the theoretical side, analyzing the underlying scheduling problem to characterize the ideal-fluid stability benchmark and the throughput gap induced by memory-growth dynamics. From an operations research perspective, several recent works analyze LLM inference scheduling under online-algorithm frameworks, focusing on competitive ratio and regret bounds. Jaillet et al. (2025) develop an online scheduling algorithm with near-optimal regret guarantees for settings with known output lengths. Wang et al. (2025) study optimization with variable prefill and decode lengths in pipeline-parallel settings. Chen et al. (2025) address robust optimization under prediction uncertainty, considering adversarial scenarios. A different theoretical angle is taken by Li et al. (2025b), whose stochastic processing model allows general batch processing times, including a piecewise-linear form that captures the compute-bound / memory-bound transition, and shows that work-conserving algorithms achieve the fluid-optimal throughput. On output length uncertainty, Jaillet et al. (2025) and Wang et al. (2025) assume lengths are known at arrival, Chen et al. (2025) considers adversarial unknown lengths, while our Nested WAIT algorithm handles stochastic unknown lengths through on-the-fly classification at segment boundaries. These works establish competitive-ratio or regret guarantees relative to hindsight benchmarks. Our

work takes a different angle: a fluid-based analysis that links the ideal-fluid stability benchmark to a memory-dependent iteration-time model (Equation (1)) and shows that threshold-based admission moves the realized stability region closer to that benchmark by preventing eviction cascades.

1.3. Notations

For integer $n \geq 1$, we denote $[n] = \{1, 2, \ldots, n\}$ as the set of integers from 1 to n. For $x \in \mathbb{R}$, denote $\lceil x \rceil$ as the smallest integer not smaller than x and $\lfloor x \rfloor$ as the largest integer not greater than x. Denote $x_+ = \max\{x, 0\}$. For set S, denote $|S|$ as its cardinality. For two functions $f(T)$ and $g(T)$, we use $f(T) = O(g(T))$ if there exists constant $c_1 > 0$ such that $f(T) \leq c_1 g(T)$ as $T \to +\infty$ and $f(T) = \Omega(g(T))$ if there exists constant $c_2 > 0$ such that $f(T) \geq c_2 g(T)$ as $T \to +\infty$.

2. Model

We model Large Language Model (LLM) inference as an online scheduling system with stochastic arrivals, staged service, and a shared memory constraint. The key departure from classical scheduling is that a job's memory requirement is not fixed at admission: as the model generates output tokens, the prompt's *Key-Value (KV) cache* grows over time. The notation is defined where it first appears, with a consolidated summary in Table 2 of Appendix B.

2.1. Prompts and Inference Process

Prompts arrive to a single Graphics Processing Unit (GPU) over time and require two phases of service. The *prefill* phase embeds the input context and initializes the prompt's KV cache. The *decode* phase then generates one output token per iteration, increasing the prompt's KV cache by one unit at each decode step. We focus on a single GPU as the basic scheduling unit; multi-GPU deployments can apply the same admission and batching logic within each GPU partition or pipeline stage.

We group prompts into m types indexed by $j \in \{1, \ldots, m\}$. Type-j prompts arrive according to a Poisson process with rate λ_j, have input length l_j after prefill, and require l'_j decode iterations before completion. Thus, a prompt of type j has the following length profile:

$$\begin{cases} \text{a length of } l_j \text{ tokens after prefill;} \\ \text{a length of } l_j + l'_j \text{ tokens after decode.} \end{cases}$$

Therefore, a type-j prompt undergoes one prefill iteration followed by l'_j decode iterations. To track service progress, we use stage $s = 0$ for prompts awaiting prefill and stages $s \in \{1, \ldots, l'_j\}$ for

prompts in decode. A prompt waiting outside the GPU at stage $s = 0$ has no resident KV cache; processing its prefill allocates l_j units, and each generated decode token adds one unit. Thus a resident type-j prompt with decode progress s occupies $l_j + s$ units of KV cache memory. This stage-dependent memory requirement is the source of the dynamic capacity constraint analyzed below.

Though the total number of different types m can be large (for example, from 1 to 10000), our algorithms and their theoretical guarantees exhibit only a weak (logarithmic) dependence on m. The analysis relies primarily on the total arrival rate across all types, ensuring that our approach scales efficiently regardless of the number of types.

While our analysis focuses on fixed arrival rates λ_j, our threshold-based algorithms can be extended to settings with time-varying arrivals by dynamically adjusting thresholds based on the arrival rates (see Appendix A).

2.2. Batching and Iteration Time

At each iteration, the scheduler forms a *batch* of prompts to process on the GPU. A batch may include newly admitted prompts in prefill and previously admitted prompts in decode, so the scheduling decision determines both how many prompts enter service and how many in-service prompts advance by one token. Because each iteration incurs a fixed launch and synchronization overhead, larger batches generally improve token throughput by spreading this overhead across more prompts. Figure 2 illustrates this timing with two sequential arrivals. Prompt $P1$ arrives first and is processed in prefill, which initializes its KV cache. Prompt $P2$ arrives later and also requires prefill. Once $P1$ has been prefilled, it enters decode and generates one output token per iteration; at that point, an iteration may process $P1$'s decode step together with $P2$'s prefill step. In later iterations, both prompts may be in decode and can be batched together. This example highlights the operational trade-off that drives the model: batching improves throughput by sharing fixed overhead across prompts, but it also increases the KV cache load carried by the system.

We next specify the service time of an iteration. In Transformer inference, prefill evaluates the input sequence and creates the initial KV cache; thereafter, each decode iteration produces one token per active prompt while attending to the prompt's cached history. For a decode-dominated batch, the aggregate active context length is therefore a natural workload descriptor. Let B^t denote the

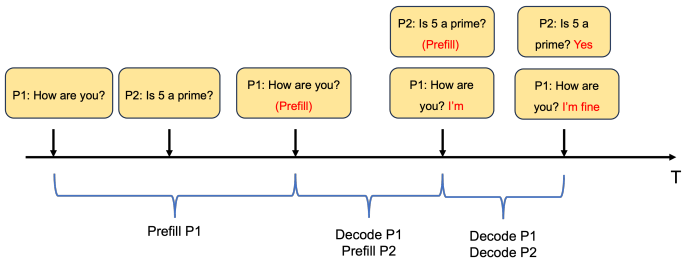

Figure 2 Example of batching and scheduling.

batch processed at iteration t, let $j(i)$ be the type of prompt i, and let s_i^t be its current stage. A prefill prompt has $s_i^t = 0$ and initializes $l_{j(i)}$ units of KV cache, while a decode prompt at stage s_i^t reads and extends a cache of size $l_{j(i)} + s_i^t$. Guided by profiling studies of LLM serving workloads (Agrawal et al. 2023, Kwon et al. 2023, Zhong et al. 2024), we approximate iteration time by

$$\tau(B^t) = d_0 + d_1 \sum_{i \in B^t} \left(l_{j(i)} + s_i^t\right), \tag{1}$$

where d_0 is the fixed per-iteration overhead and d_1 is the marginal time cost per cached token. The affine form separates this fixed overhead from the marginal cost of serving longer active contexts. It captures the admission-control trade-off central to our model: admitting more prompts spreads d_0 across more generated tokens and can therefore raise throughput, but it also lengthens subsequent iterations and consumes scarce GPU memory through larger KV caches. This regime is especially relevant because prompts typically spend most of their lifetime in decode, so admission decisions primarily determine the population and age distribution of active caches. Figure 3 validates this approximation on a single L20 GPU using Llama-2-7B and Llama-2-13B; Appendix H.2 reports the corresponding A100 calibration used in Section 6, with $d_0 \approx 276$ ms and $d_1 \approx 0.0115$ ms per KV cache token over the relevant batch-size range.

The same formulation can accommodate other calibrated service-time curves. When prefill dominates, input attention can add nonlinear length effects; mixed prefill-decode batches and hardware saturation can create distinct compute-bound and memory-bound regimes. These effects can be represented by replacing (1) with a monotone function of batch composition, including piecewise-linear specifications such as $\tau(b') = c + a \max\{0, b' - b_0\}$ (Li et al. 2025b). Prefill-decode disaggregated serving gives the complementary case: prefill work is routed to separate workers, and the decode server repeatedly executes one-token decode steps, making aggregate KV-cache usage an especially accurate state variable for iteration time (Patel et al. 2024, Zhong et al. 2024).

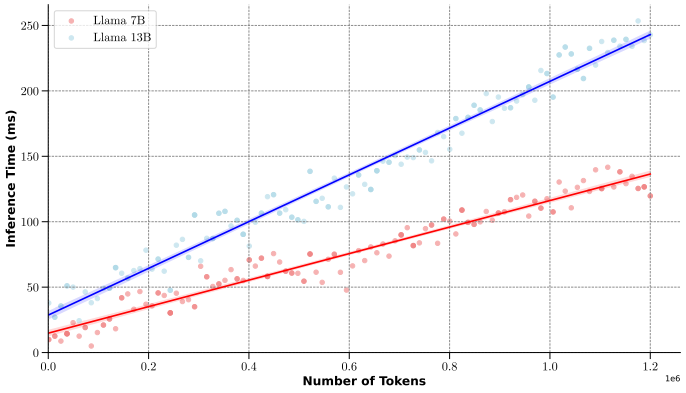

Figure 3 Inference time of Llama-2-7B and Llama-2-13B with varying number of tokens per batch on a single L20 GPU.

Thus the affine model is both a close approximation in decode-centered settings and a transparent first-order primitive for the fluid analysis. It keeps the central operational trade-off explicit: larger batches improve utilization, but they also increase iteration time and memory pressure.

2.3. Memory Constraint and Challenges

The service-time model above describes how a batch uses the GPU in one iteration. The second constraint is intertemporal: a prompt that remains in service carries its KV cache across iterations, and this cache grows as decode progresses. Hence the scheduling state must track not only the current batch B^t, but all prompts whose KV caches are currently resident on the GPU.

Memory Constraint: At all times t, the total KV cache stored on the GPU must not exceed capacity C:

$$\sum_{i \in \mathcal{G}^t} (l_{j(i)} + s_i^t) \le C, \tag{2}$$

where $\mathcal{G}^t$ is the set of prompts whose KV caches are resident on the GPU at time t. This set includes the current batch $B^t \subseteq \mathcal{G}^t$ as well as prompts that are waiting between iterations while retaining their KV caches on the GPU. Here $j(i)$ is the type of prompt i, and $s_i^t \in \{0, 1, \ldots, l'_{j(i)}\}$ is its current decode progress. Prompts that have not yet been prefilled are not in $\mathcal{G}^t$; once their KV cache is allocated, their memory contribution grows from $l_{j(i)}$ with each generated token. If the constraint (2) cannot be satisfied, the server must *evict* resident prompts by discarding their KV caches. We model eviction as last-in-first-out: the most recently admitted prompts are evicted first until enough memory is freed. An evicted prompt loses its accumulated computation and restarts from prefill, re-entering the scheduling queue as a new job. This restart rule can create an eviction cascade: each eviction relieves memory pressure temporarily, but the restarted prompts return to the

queue, raise future admission pressure, and can force additional evictions. The following example illustrates this mechanism.

EXAMPLE 1 (DYNAMIC MEMORY GROWTH AND EVICTION). Consider the running toy prompt "Hello," which receives the one-token response "Hi." This prompt type has prefill length $l = 1$ and decode length $l' = 1$. A prompt waiting for prefill uses no KV cache. Once its prefill is processed, the prompt becomes resident and uses one unit of KV cache; after it generates its single decode token, it uses $l + 1 = 2$ units just before completion. Suppose the GPU has capacity $C = 12$. One full-memory iteration can process four prefills and four decodes: the four newly prefilled prompts require $4 \times 1 = 4$ units, and the four prompts already in decode require $4 \times 2 = 8$ units, for a total of 12 units and four completions.

Now suppose three prompts are already decoding, using $3 \times 2 = 6$ units, and the scheduler admits six new prompts for prefill, using another $6 \times 1 = 6$ units during the iteration. After the iteration, the three old prompts complete and release their caches, while the six newly admitted prompts move into decode and now require $6 \times 2 = 12$ units, filling memory. If four more prompts arrive next, admitting their prefills requires four additional units. The only way to create this space is to evict two resident decode prompts, each freeing two units. These evicted prompts lose their KV caches and return to the queue as new jobs, where they compete with subsequent arrivals and can force further evictions.

This example highlights the **eviction cascade** and the admission trade-off behind it. A larger batch can raise throughput because the fixed per-iteration overhead is shared across more generated tokens, but every admitted prompt also creates a KV cache that stays on the GPU and grows one token at a time until completion. If the scheduler admits too many prompts now, later decode iterations may leave too little memory for new arrivals, forcing eviction and restart. When output lengths are unknown, this decision must be made before the scheduler knows how long each prompt will remain in memory. Section 3 formalizes the balanced admission level at which arrivals, completions, and memory growth can be kept in balance.

2.4. Objective and Policy Space

We evaluate a scheduling policy over a finite horizon $[0, T]$. The primary efficiency metric, denoted $\mathbf{Throughput}^{(T,\pi)}$, is *effective throughput*: the average number of decode tokens from prompts that complete under policy π. Tokens generated before a prompt is evicted do not count, because

eviction discards the prompt's KV cache and returns it to prefill. This convention makes memory overflow part of the performance criterion rather than a separate implementation detail. For a fixed arrival process, effective throughput is capped by the offered decode-token load. A policy falls below this cap when it leaves capacity idle or admits work that later restarts instead of completing. Admission control must therefore balance utilization against memory feasibility: admitting enough prompts to exploit the GPU, but not so many that KV-cache growth forces eviction. Maintaining this balance over a wider range of arrival rates expands the realized stability region, and under overload it keeps completed work close to the server's usable capacity. We also evaluate $\mathbf{Latency}^{(T,\pi)}$, the average elapsed time from prompt arrival to completion, and $\mathbf{TTFT}^{(T,\pi)}$, the average elapsed time from arrival to the first generated output token. These delay metrics capture service quality beyond aggregate output. In particular, TTFT rules out policies that keep completion counts high by repeatedly prioritizing short prompts while delaying the first service of longer prompts. Later sections introduce the fluid model and use it to establish throughput, latency, and TTFT guarantees for the proposed policies.

Let Π denote the class of admissible online policies. At each decision epoch t, a policy $\pi \in \Pi$ observes the prompts that have arrived, their input lengths, their current stages s_i^t, and whether their KV caches are resident on the GPU. The policy is non-anticipating: it cannot use future arrivals or a prompt's output length before that length is revealed by completion.

Given this information, the policy chooses which waiting prompts to admit, which resident prompts to batch in the current iteration, and whether to preempt or evict prompts. Preemption pauses a resident prompt while keeping its KV cache on the GPU, so it can resume without recomputation. Eviction removes the KV cache to free memory; under the LIFO rule used here, the most recently admitted resident prompts are evicted first, and each evicted prompt returns to stage $s = 0$ and must restart from prefill. All decisions must satisfy the memory constraint (2) after the resulting state transition.

3. Fluid Model and Equilibrium

Section 2 identifies the central admission-control trade-off: the scheduler should admit enough prompts to exploit batching, but not so many that their KV caches later overflow memory. This section derives the balanced operating point for that trade-off. The key state variable is the distribution of resident prompts across prefill and decode stages. As prompts move deeper into decode,

each prompt carries a larger KV cache; if too many prompts accumulate at late stages, the system can run out of memory and restart work through eviction.

We develop a fluid model that replaces stochastic sample paths with average flows and characterizes an ideal-fluid equilibrium in which arrivals, stage transitions, and completions occur at matching rates. For a given arrival vector, this equilibrium gives two reference quantities: the memory requirement M^* needed to support the equilibrium and the effective throughput $\textbf{Throughput}^*$ achieved when the offered load can be completed without eviction. Equivalently, for a fixed memory capacity C, the ideal-fluid stability region consists of arrival vectors whose equilibrium memory requirement satisfies $M^* \le C$. We derive these quantities first for a single prompt type, which isolates the stage-by-stage memory accumulation, and then for multiple prompt types sharing one memory constraint. The same equilibrium also supplies the design target for WAIT in Section 4: keep the in-service population close to the implied allocation across decode stages, thereby avoiding eviction-induced restarts while approaching the largest throughput attainable under the offered load and memory constraint.

3.1. Single-Type Fluid Model

Consider a single prompt type j. A resident prompt uses l_j units of KV cache immediately after prefill and $l_j + s$ units at decode stage $s = 1, \dots, l'_j$; after stage l'_j, the prompt completes and releases its cache. Let n^*_j denote the equilibrium number of type-j prompts at each stage. The balanced state therefore has n^*_j prompts in each of the $l'_j + 1$ stages.

Before deriving the equilibrium, we identify the load condition under which batching can be effective. Adding prompts to a batch is useful only if the additional completions outweigh the longer iteration time. Across its prefill and decode stages, a type-j prompt contributes memory-dependent work proportional to $(l'_j + 1)(l_j + l'_j/2)$. If arrivals are high enough that this memory-dependent work alone saturates the server, batching cannot restore stability.

PROPOSITION 1. *When the condition* $\lambda_j d_1 (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right) \ge 1$ *is satisfied, the system is unstable in the sense that the expected average latency under any scheduling policy with arrival rate* λ_j *grows linearly with time. Formally,*

$$\mathbb{E}[\textbf{\textit{Latency}}^{(T,\pi)}] = \Omega(T) \quad as \quad T \to \infty.$$

The proof compares arrivals and completions in a balanced batch and shows that the arrival-completion gap remains positive when the condition holds (Appendix C.1). We therefore focus on the nontrivial regime $\lambda_j d_1 (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right) < 1$, where batching can support an ideal-fluid equilibrium.

At equilibrium, the $l'_j + 1$ stages each contain n^*_j prompts, so the memory usage is

$$M^*_j = n^*_j \sum_{s=0}^{l'_j} (l_j + s) = n^*_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right), \tag{3}$$

where $l_j + l'_j/2$ is the average KV-cache usage of a type-j prompt across its prefill and decode stages.

Flow balance requires arrivals during one iteration to match completions. Since one iteration takes time $d_0 + d_1 M^*_j$, $\lambda_j (d_0 + d_1 M^*_j)$ prompts arrive and n^*_j prompts complete, giving

$$\underbrace{(d_0 + d_1 M^*_j)}_{\text{Time per iteration}} \lambda_j = \left(d_0 + d_1 n^*_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right)\right)\lambda_j = n^*_j. \tag{4}$$

Solving gives $n^*_j = d_0 \lambda_j / [1 - d_1 \lambda_j (l'_j + 1)(l_j + l'_j/2)]$. Substituting this value into the memory expression yields

$$M^*_j = n^*_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right) = \frac{d_0 \lambda_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right)}{1 - d_1 \lambda_j (l'_j + 1)(l_j + \frac{l'_j}{2})}. \tag{5}$$

The denominator is the slack in the batching feasibility condition; as it approaches zero, the inventory required for balance diverges. The balanced throughput is completed decode tokens per iteration divided by iteration time:

$$\textbf{Throughput}^*_j = \frac{n^*_j \cdot l'_j}{d_0 + d_1 n^*_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right)} = \lambda_j l'_j, \tag{6}$$

where the equality follows from the flow-balance condition (4).

3.2. Multiple-Type Fluid Model

We now extend the balance calculation to m prompt types. Type j has input length l_j, decode length l'_j, and arrival rate λ_j. Because all types draw from the same memory pool, the equilibrium is determined by the joint allocation of active prompts across decode stages and the aggregate KV-cache usage it induces.

The balance equations are written at the scheduler's iteration scale: an iteration selects a batch, advances active prompts by one stage, and updates their KV-cache usage. Let n_j^* denote the equilibrium number of type-j prompts at each stage. The resulting memory usage is

$$M^* = \sum_{j=1}^{m} n_j^*(l'_j + 1)\left(l_j + \frac{l'_j}{2}\right) \tag{7}$$

Flow balance requires $(d_0 + d_1 M^*)\lambda_j = n_j^*$ for each type j, because n_j^* type-j prompts complete in one iteration. Substituting these identities into (7) gives

$$M^* = \frac{d_0 \sum_{j=1}^{m} \lambda_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right)}{1 - d_1 \sum_{j=1}^{m} \lambda_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right)} \tag{8}$$

The corresponding iteration time is

$$\begin{aligned} \Delta T^* &= d_0 + d_1 M^* \\ &= \frac{d_0}{1 - d_1 \sum_{j=1}^{m} \lambda_j (l'_j + 1)(l_j + \frac{1}{2} l'_j)}, \end{aligned} \tag{9}$$

and the fluid throughput benchmark is

$$\textbf{Throughput}^* = \sum_{j=1}^{m} \frac{n_j^* \cdot l'_j}{d_0 + d_1 M^*} = \sum_{j=1}^{m} \lambda_j l'_j \tag{10}$$

The ideal fluid model gives an upper bound for any online policy:

PROPOSITION 2. *Suppose that the memory capacity satisfies $C \geq M^*$, where M^* denotes the memory requirement needed to support the ideal-fluid equilibrium in (8). Then, for any online policy $\pi \in \Pi$, the expected throughput satisfies*

$$\mathbb{E}[\boldsymbol{Throughput}^{(T,\pi)}] \leq \boldsymbol{Throughput}^*,$$

where $\boldsymbol{Throughput}^*$ is given in (10).

The fluid model serves two purposes. First, it provides the throughput benchmark in Proposition 2. Second, the ideal-fluid equilibrium identifies the operating point a policy should maintain: roughly n_j^* type-j prompts at each decode stage, with total memory requirement M^*. Accumulating too much late-stage work consumes memory and risks eviction, while holding too little late-stage work leaves completions below the fluid rate. Section 4 uses M^* and n_j^* to define thresholds that keep the decode-stage populations near the ideal-fluid equilibrium.

4. Scheduling with Known Arrival Types: The WAIT Algorithm

This section develops the *Waiting for Accumulated Inference Threshold* (WAIT) algorithm for the case in which prompt types and arrival rates are known. WAIT uses the ideal-fluid equilibrium from Section 3 as a batching target: for each type j, it waits until enough prompts have accumulated before admitting a batch, so that the active decode-stage populations remain close to the equilibrium levels n_j^*. This threshold rule prevents eviction cascades while allowing the server to approach the fluid throughput benchmark. The throughput guarantees below assume $C \geq M^*$, where M^* is the memory requirement needed to support that ideal-fluid equilibrium in (8); this is the feasibility condition under which the equilibrium allocation across decode stages fits in memory. The same load-balancing and non-eviction logic also applies under overload, where the goal is to exploit the available system capacity as effectively as possible; Section 6 illustrates this behavior numerically.

4.1. Motivation: Why Naive Scheduling Fails

The condition $C \geq M^*$ is a capacity condition, not a scheduling rule. It identifies when the ideal-fluid equilibrium can fit in memory, but it does not ensure that an online policy will keep work balanced across decode stages.

Consider a First-Come-First-Serve (FCFS) policy that admits prompts as soon as they arrive. Even when $C = M^*$, so the ideal-fluid equilibrium fits exactly in memory, FCFS can move too much work into downstream decode stages and create a persistent throughput loss. The following proposition gives a worst-case lower bound for this failure mode.

PROPOSITION 3. *When $C = M^*$, there exists an instance in which First-Come-First-Serve (FCFS) incurs a throughput gap $\boldsymbol{Throughput}^* - \mathbb{E}[\boldsymbol{Throughput}^{(T,\pi)}] = \Omega(1)$ as $T \to \infty$.*

The proof is provided in Appendix C.3. The example below is a simpler single-type illustration of the same eviction cascade.

EXAMPLE 2 (EVICTION CASCADE UNDER FCFS). Consider a single prompt type: a prompt such as "Hello" with a one-token response "Hi," so $l = 1$ and $l' = 1$. Each prompt uses 1 unit of memory after prefill and 2 units after decode. Let $C = 12$ and $\lambda = 4$. The ideal-fluid model keeps 4 prompts at each stage, uses $4 \times 1 + 4 \times 2 = 12$ units of memory, and completes 4 prompts per iteration.

Now suppose the system starts slightly imbalanced with 6 prompts at stage 0 (awaiting prefill) and 3 prompts at stage 1 (in decode). Under FCFS:

1. **Iteration 1**: FCFS processes all 9 prompts. The 3 decode prompts complete, while the 6 prefill prompts advance to decode and require $6 \times 2 = 12$ units of memory. The system is full after only 3 completions.
2. **Iteration 2**: About 4 new prompts arrive during the previous iteration. Admitting them for prefill requires 4 additional memory units, so the server evicts 2 decode prompts to free $2 \times 2 = 4$ units. The system returns to 4 prefill prompts and 4 decode prompts, again using $4 + 8 = 12$ units.
3. **Iteration 3**: The 4 decode prompts complete. The 2 evicted prompts have lost their KV caches and must restart from prefill, re-entering the queue alongside genuinely new arrivals. The restart mechanism perpetuates the imbalance.

The eviction-restart cycle yields approximately 3–3.5 completions per iteration instead of the 4 completions sustained by the ideal-fluid equilibrium. Thus $C = M^* = 12$ is sufficient to support the ideal-fluid balance, but FCFS does not maintain that balance and loses effective throughput.

The operational implication is that having enough memory to support the ideal-fluid equilibrium does not by itself guarantee that the stochastic system will operate near that equilibrium. The scheduler must also regulate the decode-stage mix. FCFS admits prompts greedily, lets the allocation of active prompts across decode stages drift away from the ideal-fluid balance, and thereby creates the eviction-restart cycle.

4.2. The WAIT Algorithm

The FCFS example points to the missing control variable: the composition of the in-service population across decode stages. WAIT regulates this composition by assigning each type j a threshold n_j, interpreted as the target number of type-j prompts to serve from each decode stage in an iteration. If fewer than n_j new type-j prompts are waiting at entry, WAIT does not admit

that type. Once the threshold is reached, the server processes up to n_j prompts from each decode stage of that type. In this way, new work enters the system only when it can replenish a balanced allocation across decode stages, rather than pushing the system toward the downstream buildup seen under FCFS.

The threshold rule also gives the analysis its tractable structure. Once the thresholds are fixed, each type evolves as a threshold queue observed at batch-completion epochs, while the shared memory interaction is captured by the feasibility of the threshold vector. The waiting requirement is therefore not merely a safety device against eviction; it also preserves the batching benefit created by the fixed overhead d_0. Batches form at a scale large enough to use the server effectively, while the thresholds keep memory usage within capacity.

Algorithm Description. For each prompt type $j \in [m]$, WAIT sets a threshold n_j derived from the fluid model. At decision epoch t, let n^t_{js} denote the number of type-j prompts at stage s. Type j is eligible for the next batch only when

$$n^t_{j0} \geq n_j.$$

When this condition holds, WAIT selects $\min\{n_j, n^t_{js}\}$ type-j prompts from each stage s. If no type is eligible, the server waits for more arrivals. Prompts that are not selected remain in their current queues; those already in decode keep their KV caches intact. Algorithm 1 and Figure 4 summarize the policy.

4.3. Asymptotic Guarantees

We analyze WAIT in a long-horizon scaling that averages out stochastic fluctuations while preserving the fixed memory constraint. For $\zeta \geq 1$, arrival rates scale as $\lambda^{(\zeta)}_j = \zeta\lambda_j$ and processing times scale as $(d^{(\zeta)}_0, d^{(\zeta)}_1) = (\zeta^{-1}d_0, \zeta^{-1}d_1)$, with memory capacity C fixed. For a policy π, let $N^{(\zeta,\pi)}_j$ be the number of completed decode tokens from type-j prompts over this horizon, and define

$$\textbf{Throughput}^{(\zeta,\pi)} = \frac{1}{\zeta T}\sum_{j=1}^{m} N^{(\zeta,\pi)}_j.$$

For delay metrics, let $\textbf{Latency}^{(\zeta,\pi)}_{\text{cal}}$ and $\textbf{TTFT}^{(\zeta,\pi)}_{\text{cal}}$ denote the elapsed calendar-time averages over this scaled system. The theorem reports the service-normalized delays $\textbf{Latency}^{(\zeta,\pi)} := \zeta\textbf{Latency}^{(\zeta,\pi)}_{\text{cal}}$ and $\textbf{TTFT}^{(\zeta,\pi)} := \zeta\textbf{TTFT}^{(\zeta,\pi)}_{\text{cal}}$, so the effective horizon in the bounds is ζT. The benchmark is the fluid throughput $\textbf{Throughput}^* = \sum_{j=1}^{m} \lambda_j l'_j$ in (10).

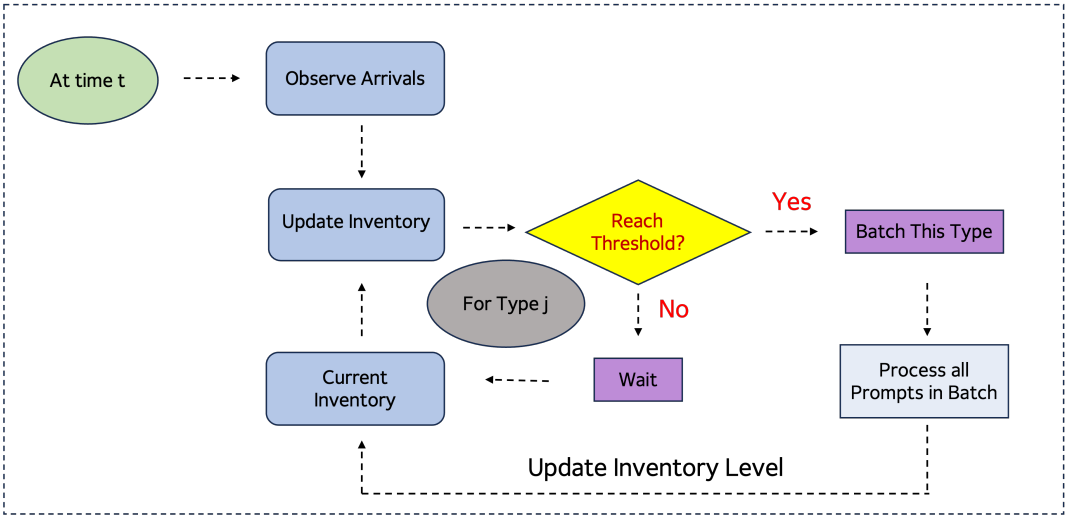

Figure 4 Batch formation under WAIT for a fixed type j. The scheduler updates the type-j inventory and batches this type only when the entry-stage queue reaches n_j; otherwise, it waits and carries the inventory forward. When batched, up to n_j prompts from each stage are advanced.

Algorithm 1 WAIT: Waiting for Accumulated Inference Threshold

Input: Memory C, arrival rates λ_j, thresholds n_j satisfying (11), $\forall j \in [m]$

Initialize prompt inventory $n_{js} \leftarrow 0$ for all $j \in [m], s \in \{0, 1, \ldots, l'_j\}$
Initialize event queue with arrival events for each type j
Set current time $t \leftarrow 0$
while True **do**
  Wait for the next event (arrival or batch completion)
  if event is an arrival of type j **then**
    Update inventory: $n_{j0} \leftarrow n_{j0} + 1$ ▷ Add new prompt to waiting queue of prefill phase
  else if event is a batch completion **then**
    For each j in the completed batch and stage s, let $a_{js} \leftarrow \min\{n_j, n_{js}\}$ be the selected type-j prompts
    Advance selected prompts one stage: $n_{js} \leftarrow n_{js} - a_{js}$ and $n_{j,s+1} \leftarrow n_{j,s+1} + a_{js}$ for $s < l'_j$
    Clear KV caches for the a_{j,l'_j} prompts completing after stage l'_j ▷ Free memory for completed prompts
  end if
  Check if $n_{j0} \geq n_j$ for any $j \in [m]$ ▷ Check threshold for batching
  if condition $n_{j0} \geq n_j$ is met for some j **then**
    Form batch B by selecting $\min\{n_j, n_{js}\}$ prompts of type j at each stage s for all j meeting the condition
    Process batch B, keep prompts outside the batch waiting and do not delete their KV caches
  end if
end while

Assume $C \geq M^*$, with M^* given by (8). Consider a WAIT policy $\pi = [n_1, \ldots, n_m]$, where n_j is the type-j threshold applied at each stage. The threshold vector must satisfy a load-balance condition

and a memory condition:

$$\begin{aligned} \Delta T(n_{1:m}) &:= d_0 + d_1 M^\pi \le \tfrac{n_j}{\lambda_j},\ \forall j \in [m], \\ M^\pi &= \sum_{j=1}^{m} n_j (l'_j + 1)\big(l_j + \tfrac{l'_j}{2}\big), \\ M^* &\le M^\pi \le C \end{aligned} \tag{11}$$

The first line is the arrival-completion balance for each type: during an iteration of length $\Delta T(n_{1:m})$, the expected number of type-j arrivals is $\lambda_j \Delta T(n_{1:m})$, and WAIT serves n_j type-j prompts from each active stage. The memory condition then asks the threshold vector to be large enough to support the ideal-fluid equilibrium, but not so large that a threshold-sized batch can exceed the available memory C.

THEOREM 1. *If the thresholds $\pi = [n_1, \dots, n_m]$ satisfy (11), then*

$$\begin{aligned} &\boldsymbol{Throughput}^* - \mathbb{E}\left[\boldsymbol{Throughput}^{(\zeta,\pi)}\right] = O((\zeta T)^{-\frac{1}{2}}), \\ &\mathbb{E}\left[\boldsymbol{Latency}^{(\zeta,\pi)}\right], \mathbb{E}\left[\boldsymbol{TTFT}^{(\zeta,\pi)}\right] = O\left((\zeta T)^{\frac{1}{2}}\right). \end{aligned}$$

Moreover, if $\Delta T(n_{1:m}) < n_j/\lambda_j$ for all $j \in [m]$, then

$$\begin{aligned} &\boldsymbol{Throughput}^* - \mathbb{E}\left[\boldsymbol{Throughput}^{(\zeta,\pi)}\right] = O(1/(\zeta T)), \\ &\mathbb{E}\left[\boldsymbol{Latency}^{(\zeta,\pi)}\right], \mathbb{E}\left[\boldsymbol{TTFT}^{(\zeta,\pi)}\right] = O(1). \end{aligned}$$

The theorem rests on the control structure created by WAIT. The difficult step is not to prove recurrence after a stable queue has been isolated. Instead, it is to design a batching rule that keeps KV-cache growth under control before memory overflows. Each admitted prompt creates a KV cache that grows throughout decode. If an unstructured batching rule admits too much work, these caches can exceed memory in later iterations, forcing eviction and restart. The restarted prompts then return as future work and can reduce effective throughput even when the ideal-fluid memory requirement fits within capacity. WAIT prevents this failure mode by fixing a target allocation across decode stages through $n_{1:m}$. Given this threshold vector, every full batch has memory usage M^π and iteration length $\Delta T(n_{1:m})$, and the main stochastic component for type j is the time until its entry queue reaches n_j. The proof then works with the induced threshold queues: the load-balance condition gives nonpositive drift for each queue, while the memory condition prevents threshold-sized batches from exceeding capacity. In this sense, WAIT turns the original memory-coupled

controlled process into threshold queues tied together by the deterministic feasibility condition $M^\pi \le C$. The complete proof is in Appendix D.

The square-root delay bound at the boundary is also unavoidable. When the threshold queues have no slack, their fluctuations cannot be averaged away by any non-predictive online rule, even if the total memory equals the ideal-fluid memory requirement. The next proposition gives the matching lower bound for latency and TTFT.

PROPOSITION 4. *There exists an instance where $C = M^*$, and for any non-predictive online policy π, the expected average latency and TTFT satisfy*

$$\mathbb{E}[\boldsymbol{Latency}^{(\zeta,\pi)}], \quad \mathbb{E}[\boldsymbol{TTFT}^{(\zeta,\pi)}] = \Omega((\zeta T)^{1/2}).$$

The strict inequalities in Theorem 1 correspond to slack in these threshold queues. If $\Delta T(n_{1:m}) < n_j/\lambda_j$, the type-$j$ entry queue has negative drift, and the average latency and TTFT of type-j prompts remain $O(1)$.

The policy in this section assumes that type labels and arrival rates are known when the thresholds are set. In practice, these quantities can be estimated from predictors (Fu et al. 2025), but an online serving system may not know a request's decode length at admission. The next section relaxes this information requirement by developing a nested version of WAIT for unknown prompt types.

5. Scheduling with Unknown Arrival Types: The Nested WAIT Algorithm

The WAIT algorithm assumes that prompt types are known when thresholds are set. This section removes that information assumption. The *Nested Waiting for Accumulated Inference Threshold* (Nested WAIT) algorithm organizes decoding into nested segments and uses output-length information only as it is revealed by the prompt's own decode trajectory.

5.1. Algorithm Overview

We first return to the running example to show how unknown output lengths complicate policy design and memory control. The same admitted prompts may either finish at an early decode boundary or continue to later stages with larger KV caches, so neither an optimistic nor a conservative admission rule is satisfactory.

EXAMPLE 3 (UNKNOWN OUTPUT LENGTHS). Continuing Example 2, suppose output lengths are now *unknown* at arrival: each "Hello" prompt receives either "Hi" (1 token) or "Hi there!" (2 tokens). The scheduler faces a dilemma. If it optimistically treats all outputs as short, then capacity $C = 12$ appears to allow 6 prompts, because each would use 2 units after one decode token. If some prompts instead generate two tokens and require 3 units each, the same admission decision can overflow memory and trigger eviction. A conservative rule that treats every prompt as long avoids this overflow, but admits only 4 prompts and wastes capacity whenever most prompts are short.

Nested WAIT avoids both extremes by pooling prompts in early decode stages and separating them only at completion boundaries. Segment 1 serves all prompts until the first possible output length l'_1; prompts that complete there leave the system and release memory, while the remaining prompts enter Segment 2. The same logic repeats across later boundaries. The policy therefore does not reserve memory for long outputs before they are observed, but still lets short prompts complete without waiting behind longer ones.

Formally, let output lengths satisfy $l'_1 < l'_2 < \cdots < l'_m$, where type j has decode length l'_j, and set $l'_0 = 0$. Nested WAIT defines m segments, with segment k covering the decode work between boundaries l'_{k-1} and l'_k. The segment receives types $k, k+1, \ldots, m$, whose combined arrival rate is $\lambda_k + \cdots + \lambda_m$. The threshold vector $\pi = [n_1, \ldots, n_m]$ has two roles: n_k caps the number of prompts processed per stage in segment k, and it is also the number of prompts that must be available at the segment's entry boundary before that segment begins service.

Thus, if segment k^* has fewer than n_{k^*} prompts at its entry boundary, Nested WAIT pauses segment k^* and all later segments while retaining their KV caches on the GPU. Earlier segments $k < k^*$ can continue batching up to n_k prompts per stage, so new arrivals can keep moving through the shared early stages. Once enough prompts reach the boundary of segment k^*, that segment resumes. Algorithm 2 formalizes this procedure, and Figure 5 illustrates the resulting pipeline.

Nested WAIT's segmentation rule is based on decode progress. For the main analysis, we take a common prefill length l to keep the memory expressions transparent; Appendix A discusses heterogeneous prefill lengths and coarser segmentations.

In the two-type instance from Example 3, Nested WAIT uses two segments. Segment 1 processes all newly arrived prompts through stages 0 and 1 at combined rate $\lambda_1 + \lambda_2$; type-1 prompts then

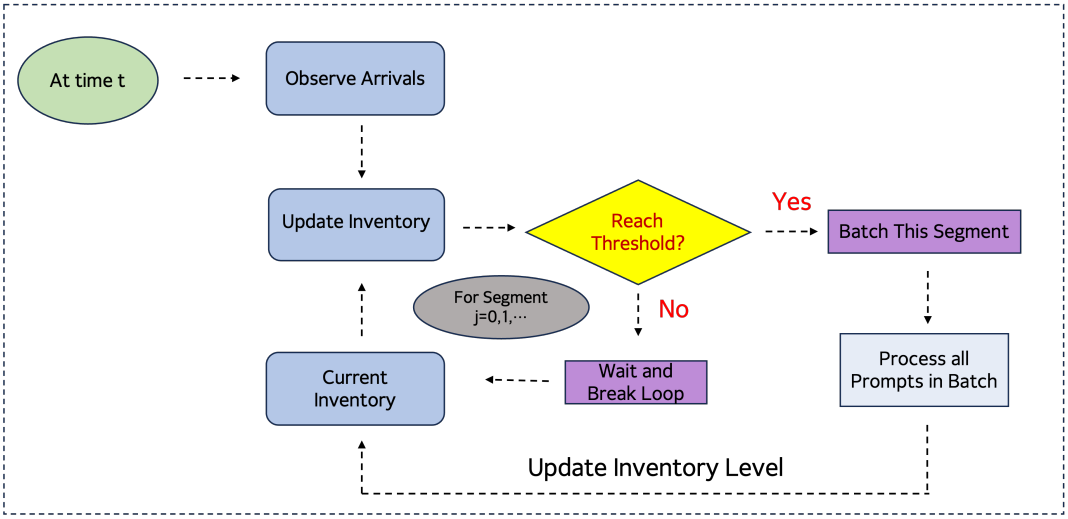

Figure 5 Nested WAIT pipeline. Prompts enter the first segment without output-length labels; short prompts complete and exit at early boundaries, while longer prompts advance to later segments where separate thresholds regulate batching.

complete, while type-2 prompts enter Segment 2 and are processed at rate λ_2. Figure 6 shows this operation over time. Segment 1 waits until enough newly arrived prompts have accumulated, and Segment 2 waits until enough prompts have survived the first segment. When both thresholds are met, the active segments are scheduled together. Thus early segments provide shared batching, while later segments regulate service for prompts that have been identified as longer jobs.

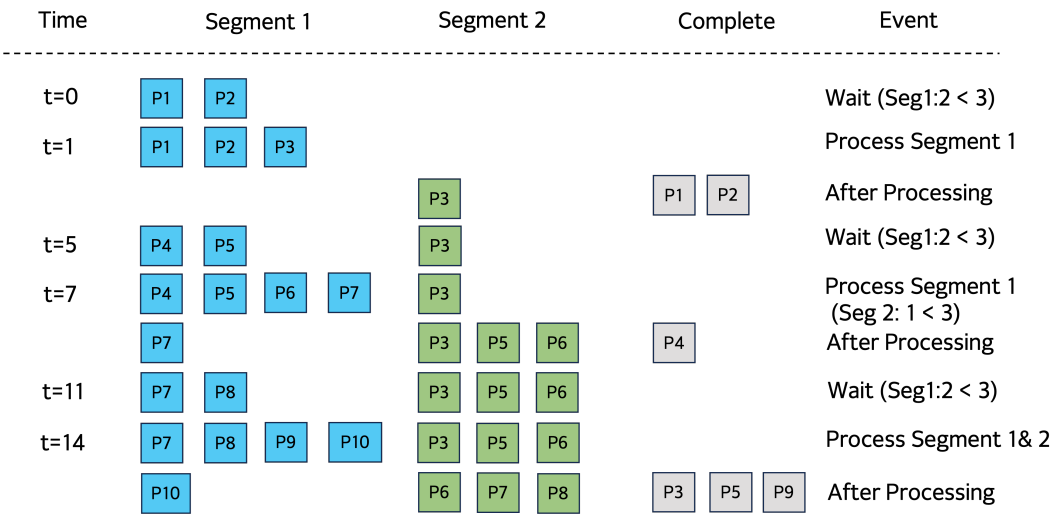

Figure 6 Two-segment Nested WAIT example. Segment 1 batches newly arrived prompts once its threshold is met; Segment 2 starts only after enough prompts have survived the first segment and revealed themselves as longer jobs.

Algorithm 2 Nested WAIT: Nested Waiting for Accumulated Inference Threshold

Input: Memory capacity C, arrival rates λ_j for $j \in [m]$, thresholds n_k for segments $k \in [m]$
Input: Output lengths $l'_1 < l'_2 < \cdots < l'_m$ for prompt types $j \in [m]$
▷ $Q_{k,s}$: number of prompts in segment k at decode stage s; entry stage of segment k is l'_{k-1} (with $l'_0 = 0$)
Initialize $Q_{k,s} \leftarrow 0$ for all segments $k \in [m]$ and stages $s \in \{l'_{k-1}, l'_{k-1}+1, \ldots, l'_k - 1\}$
Initialize event queue; set current time $t \leftarrow 0$
while True **do**
  Wait for the next event (arrival or batch completion)
  if event is an arrival **then**
    $Q_{1,0} \leftarrow Q_{1,0} + 1$ ▷ New prompts enter segment 1 at stage 0
  else if event is a batch completion for segment k **then**
    for each stage s in segment k **do**
      Advance $\min\{n_k, Q_{k,s}\}$ prompts from stage s to $s+1$
    end for
    if prompts reach stage l'_k (end of segment k) **then**
      if $k < m$ **then**
        Move to segment $k+1$ at stage l'_k ▷ These are type-$(k+1)$ or longer
      else
        Complete and clear KV caches
      end if
    end if
  end if
  Find largest k such that $Q_{k',l'_{k'-1}} \geq n_{k'}$ for all $k' \leq k$ ▷ All segments up to k meet threshold
  if such k exists **then**
    Form batch from segments $1, \ldots, k$: select $\min\{n_{k'}, Q_{k',s}\}$ prompts per stage
    Process batch; prompts outside batch wait with KV caches retained
  end if
end while

Algorithm 2 gives the formal description of Nested WAIT. Completion events determine which prompts exit and which continue downstream; thresholds determine when each active segment has enough work to process without sacrificing batching efficiency.

5.2. Main Theorem

The analysis uses the same long-horizon scaling and scaled delay metrics as Section 4: arrivals and service speed scale with ζ, memory capacity remains fixed, and the effective horizon in the bounds is ζT. The new difficulty is that downstream segments receive endogenous input. The prompts entering segment k are exactly those that survived earlier segments, so their arrivals depend on upstream batching, completion events, and the resident KV caches carried across segment boundaries. We study throughput $\mathbf{Throughput}^{(\zeta,\pi)}$, latency $\mathbf{Latency}^{(\zeta,\pi)}$, and time to first token $\mathbf{TTFT}^{(\zeta,\pi)}$ under a threshold vector $\pi = [n_1, \ldots, n_m]$.

The need for additional memory is already visible at $C = M^*$. At this capacity, the ideal-fluid equilibrium fits in memory if types are known. A non-predictive policy, however, must admit prompts before knowing whether they will finish at an early boundary or continue downstream.

Optimistic admission can overflow memory once prompts reveal longer outputs; conservative admission avoids that risk but leaves capacity unused. The resulting uncertainty creates a non-vanishing throughput loss.

PROPOSITION 5. *When memory capacity is exactly at the ideal-fluid stability boundary, $C = M^*$, there exists an instance where any online non-predictive policy π, unaware of prompt types upon arrival, incurs a throughput gap* $\boldsymbol{Throughput}^* - \mathbb{E}[\boldsymbol{Throughput}^{(\zeta,\pi)}] = \Omega(1)$.

This lower bound shows that uncertainty has a memory cost: M^* supports the ideal-fluid equilibrium with known types, but does not provide enough room for the boundary queues created by unknown output lengths. Theorem 2 shows that Nested WAIT closes this gap with a logarithmic safety buffer. The construction has two parts: choose thresholds that give these boundary queues negative drift, and reserve enough memory for their residual stochastic fluctuations.

The threshold vector must make each segment queue stable while keeping batches large enough to exploit the GPU. Let $\Delta T_{[1,\dots,m]}(n_1,\dots,n_m)$ denote the iteration time when segment k contains exactly n_k prompts per stage for each $k \in [m]$. The required drift conditions are

$$\begin{aligned} &\Delta T_{[1,\dots,m]}(n_1,\dots,n_m) < \frac{n_1}{\sum_{j=1}^m \lambda_j}, \\ &\frac{n_k}{n_{k-1}} > p_k, \forall k \in \{2,\dots,m\}, \end{aligned} \tag{12}$$

where $p_k = (\sum_{j=k}^m \lambda_j)/(\sum_{j=k-1}^m \lambda_j) < 1$ for $k \geq 2$. The first inequality gives negative drift for the entry queue of Segment 1; the second gives negative drift for each downstream boundary queue, since only a p_k fraction of prompts leaving segment $k-1$ continue to segment k. For memory accounting, define

$$\delta_1 = l_1' + 1, \quad \bar{s}_1 = \frac{l_1'}{2}, \qquad \delta_k = l_k' - l_{k-1}', \quad \bar{s}_k = \frac{l_{k-1}' + 1 + l_k'}{2} \quad (k \geq 2).$$

The base memory used by threshold-sized segment batches is

$$M^\pi = \sum_{k=1}^m n_k (l + \bar{s}_k)\delta_k. \tag{13}$$

Here δ_k is the number of stages covered by segment k, and $\bar{s}_k$ is the average decode progress across those stages. The term M^π is the threshold analogue of the ideal-fluid memory requirement M^*: if n_k is chosen on the scale of the cumulative fluid population that survives to segment k,

then M^{π} is on the same scale as M^{*}, up to rounding and segment aggregation. The additional terms in the theorem are the memory buffers for downstream boundary queues created by unknown output lengths. Appendix A.2, especially Figures 14–16, gives a numerical decomposition showing this base term relative to the safety buffers. With these definitions, the theorem below gives the performance guarantee.

THEOREM 2. *Assume thresholds* $\pi = [n_1, \ldots, n_m]$ *satisfy* (12)*, and memory* M *satisfies*

$$\begin{aligned} & M^{(\zeta,\pi)} \\ \geq & M^{\pi} + \sum_{k=2}^{m} (l + l'_{k-1}) \left(n_k + \theta_k^{-1} \ln \left(\frac{m\zeta T}{\delta(d_0 + d_1 M^{\pi})} \right) \right) \\ = & O\left(2M^{\pi} + \sum_{k=2}^{m} (l + l'_{k-1}) \theta_k^{-1} \ln \left(\frac{m\zeta T}{\delta(d_0 + d_1 M^{\pi})} \right) \right), \end{aligned} \tag{14}$$

where $\theta_k \geq 8(n_k - n_{k-1}p_k)/n_{k-1}$ *for* $k \geq 2$*. Then:*

$$\begin{aligned} & \textbf{\textit{Throughput}}^{*} - \mathbb{E}\left[\textbf{\textit{Throughput}}^{(\zeta,\pi)}\right] = O\left((\zeta T)^{-1}\right), \\ & \mathbb{E}\left[\textbf{\textit{Latency}}^{(\zeta,\pi)}\right], \mathbb{E}\left[\textbf{\textit{TTFT}}^{(\zeta,\pi)}\right] = O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ *over horizon* $[0, T]$.

Unknown output lengths make this memory-control problem harder. At admission, the scheduler does not know which prompts will finish early and which will continue into later, more memory-intensive decode stages. Yet memory growth starts as soon as a prompt's KV cache becomes resident on the GPU. As Example 3 illustrates, prompts reveal their output lengths only by either completing at a boundary or continuing beyond it. Completed prompts leave and release memory, while surviving prompts carry larger KV caches downstream. Without segment-level control, these surviving prompts can create late-stage buildup, overflow memory, and trigger eviction-induced restarts. Nested WAIT therefore regulates both admission and stage advancement while the type mix is still being revealed.

The proof follows the prompts at these segment boundaries, where output-length information is revealed and downstream memory growth must be controlled. Conditional on a threshold-sized batch leaving segment $k-1$, the number that continue to segment k follows binomial thinning with probability p_k. This coupling turns the unknown-output-length process into boundary queues. These queues track how much surviving work is allowed to continue into later segments, which is

the part of the memory-growth trace that can cause overflow. In the memory condition (14), M^{π} covers threshold-sized segment batches, and the logarithmic term is the safety buffer that keeps retained KV caches at downstream boundaries from causing overflow and eviction. The theorem states the clean case with one boundary per output length. In practice, the number of possible output lengths can be large, and the same construction can be implemented with coarser decode segments that group nearby lengths and maintain thresholds only at the segment boundaries. Appendix A discusses this general segment design, and Section 6 evaluates it in simulated serving workloads.

6. Numerical Experiments

We evaluate the WAIT and Nested WAIT algorithms on two classes of workloads: synthetic Poisson arrivals (Section 6.1) and a real prompt distribution sampled from lmsys-chat-1m (Section 6.2). On every workload we benchmark our algorithms against two widely deployed inference schedulers, vLLM (Kwon et al. 2023) and Sarathi (Agrawal et al. 2023). Both are FCFS-based serving policies in our experiments; Sarathi differs by splitting prefill into chunks so that a long prefill does not monopolize an iteration that could also serve decode work. We use Vidur (Agrawal et al. 2024a) as the discrete-event simulator, configured to model a single NVIDIA A100 80 GB serving Llama-2-7B. Vidur's iteration-time predictions agree with direct SGLang measurements on a physical A100 within a mean absolute percentage error (MAPE) of 1.93% on a single-type calibration grid with batch sizes $1, 2, 4, \ldots, 256$. The workloads below are designed to stress decode-side KV-cache growth, which is the regime targeted by the affine iteration-time approximation in Section 2; Appendix H reports supplemental simulator validation and end-to-end real-GPU runs of WAIT.

With exogenous arrivals at rate λ, a stable policy completes requests at that rate in steady state, so completion-rate differences appear only when a policy approaches or exceeds the arrival rates it can sustain. The main figures plot mean end-to-end latency against λ, averaging each point over 10 independent simulation replications with different arrival draws. We use a policy's empirical stability boundary λ^* to denote the largest tested arrival rate at which the latency curve remains controlled and the effective-completion-rate curve still tracks the offered load. Above that range, the effective completion rate saturates or declines because the policy suffers overload or eviction-induced restarts. The reported latency at such rates summarizes the completed requests observed in the simulation run, while the completion-rate panel shows that the policy no longer keeps pace with the offered load.

Hardware. All experiments use Vidur to simulate a single NVIDIA A100 80 GB serving Llama-2-7B. The iteration-time model is Equation (1); its A100 calibration and GPU-side validation are reported in Appendix H.2.

Baseline configurations (vLLM, Sarathi). Both baselines use their default Vidur configurations and admit arrivals in FCFS order. They also use the same local capacity-reservation rule: before admitting a new request, the scheduler requires a small memory buffer (1% of KV-cache capacity) to remain unused. This rule protects against admitting a request when the server is already essentially full, but it is not a dynamic limit on the total number of jobs in service. Once admitted requests continue decoding, their KV caches can still grow beyond the available capacity, in which case the scheduler evicts and restarts an in-progress request. The difference between the baselines is in how prefill is batched. vLLM processes a newly admitted request's full prefill in one iteration. Sarathi bounds the prefill work per iteration to chunks of at most 512 tokens, allowing partial prefills to be interleaved with decode iterations. This chunked-prefill rule reduces head-of-line blocking from long prefills, but it does not control the cumulative decode-side population. Thus neither baseline enforces the load-balancing threshold used by WAIT.

WAIT and Nested WAIT settings. In the experiments below, the main tuning variable for WAIT and Nested WAIT is a system-wide batch-size cap tl, which controls how many requests can be served concurrently across decode stages. This cap is not itself one of the per-stage scheduling thresholds. In a single-type WAIT run, tl is distributed across the decode stages of that workload, yielding the integer per-stage thresholds used by the scheduler. In a Nested WAIT run, tl is first allocated across decode-stage segments according to the segment lengths and continuation probabilities, and the resulting segment-level caps are then spread across the stages inside each segment. For each workload, the corresponding subsection specifies the tuning grid, segment partition, and threshold construction used in the experiment. Under the baseline settings used for the arrival-rate figures, WAIT and Nested WAIT do not trigger eviction.

6.1. Synthetic Workloads

We consider three synthetic Poisson workloads. The single-type workload `p512d20` has prefill length 512 and decode length 20, isolating the effect of WAIT's threshold mechanism (Algorithm 1). The two-type workload (0.7 `p512d20`, 0.3 `p512d50`), meaning that each arrival is `p512d20` with probability 0.7 and `p512d50` with probability 0.3, tests the segment design of Nested WAIT

(Algorithm 2) on prompts that share the prefill profile but differ in decode length. The long-decode workload `p512d1000` keeps the same prefill length but increases the decode length to 1000, allowing us to examine the same threshold mechanism when KV cache growth is much more pronounced.

Single-type workload (WAIT). For this workload, WAIT chooses tl from the finite grid $20, 25, \ldots, 50$ at each arrival rate and converts it into the corresponding per-stage limits. Figure 7 shows mean end-to-end latency (left, on a stretched-symlog scale that expands the stable region) and effective completion rate (right), i.e., completed requests per second net of eviction-induced restarts, versus arrival rate λ. Because all requests are homogeneous, this workload highlights the effect of threshold-based admission without the additional segment structure used in Nested WAIT. WAIT keeps latency controlled up to $\lambda \approx 23$, whereas Sarathi and vLLM lose stability near $\lambda \approx 22$ and $\lambda \approx 19$, respectively. At low load the three policies are close, but the gap widens rapidly as λ approaches the boundary: WAIT attains the smallest mean latency at every tested rate in $\lambda \in \{12, \ldots, 24\}$, with the gap to Sarathi growing from ~2% at $\lambda = 12$ to over 70% at $\lambda = 23$. This pattern matches the paper's mechanism. Because there is only one request type, the gain does not come from type separation; it comes from threshold-based admission itself, which keeps the in-system population controlled and delays the onset of instability. The right panel shows the same insight in completion-rate terms: WAIT follows the ideal λ-line up to a higher arrival rate than Sarathi and vLLM.

Two-type workload (Nested WAIT). For the two-type workload, Nested WAIT uses the same system-wide batch-size cap tl and two decode-stage segments, 1–20 and 21–50. The second segment receives the prompts that survive the first 20 decode stages, so its continuation ratio is 0.3. Thus the implementation sets real-valued per-stage thresholds in the ratio $n_2/n_1 = 0.3$ and solves $20n_1 + 30n_2 = \text{tl}$ before rounding the corresponding segment-level caps to integers; for example, tl = 30 gives segment-level caps approximately 21 and 9. Figure 8 shows the same comparison for the two-type workload ($0.7\,$`p512d20`$, 0.3\,$`p512d50`). The stability ordering is again vLLM first ($\lambda^* \approx 18$), Sarathi second ($\lambda^* \approx 21$), and Nested WAIT last ($\lambda^* \approx 22$). Nested WAIT attains the lowest mean latency over these tested rates, with the gap to Sarathi growing from ~6% at low load to over 40% near the boundary. The mechanism also differs from the single-type case. Here the two request classes have the same prefill length and differ only in decode length, so the main source of congestion is the accumulation of the long-decode minority in later decode stages. Nested WAIT's segment thresholds keep that downstream inventory from expanding too far, which prevents those

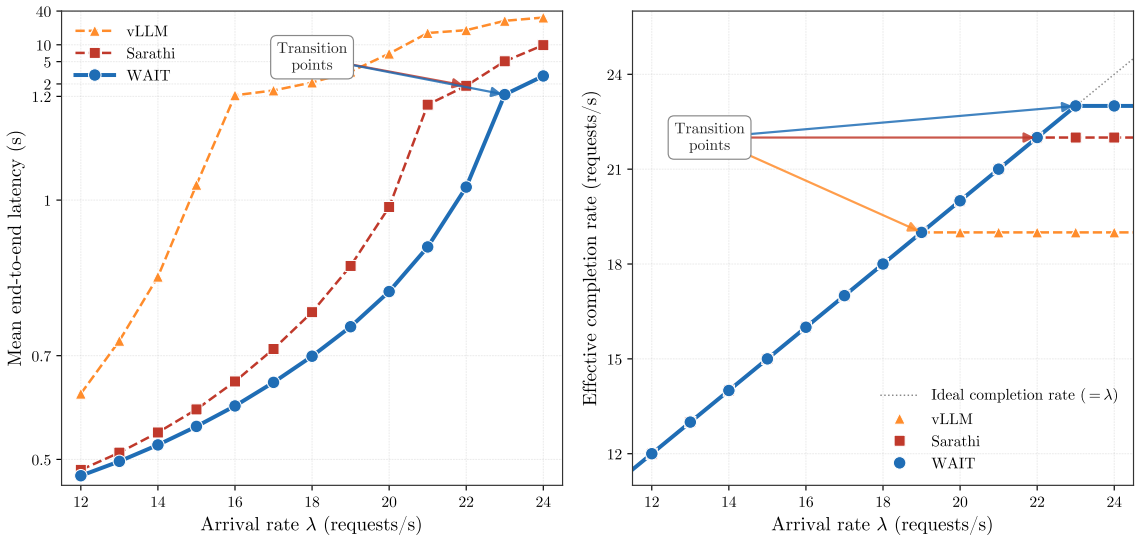

Figure 7 Single-type workload (`p512d20`, Poisson arrivals): mean end-to-end latency (left) and effective completion rate (right) versus arrival rate λ. The arrows mark each policy's stability boundary λ^*; the dotted line in the right panel indicates the ideal completion rate equal to λ.

delays from propagating to the short-decode majority. The resulting gain is therefore not only at entry: by controlling downstream buildup, Nested WAIT preserves low latency for the short jobs and enlarges the stable operating region beyond what a single global cap can achieve.

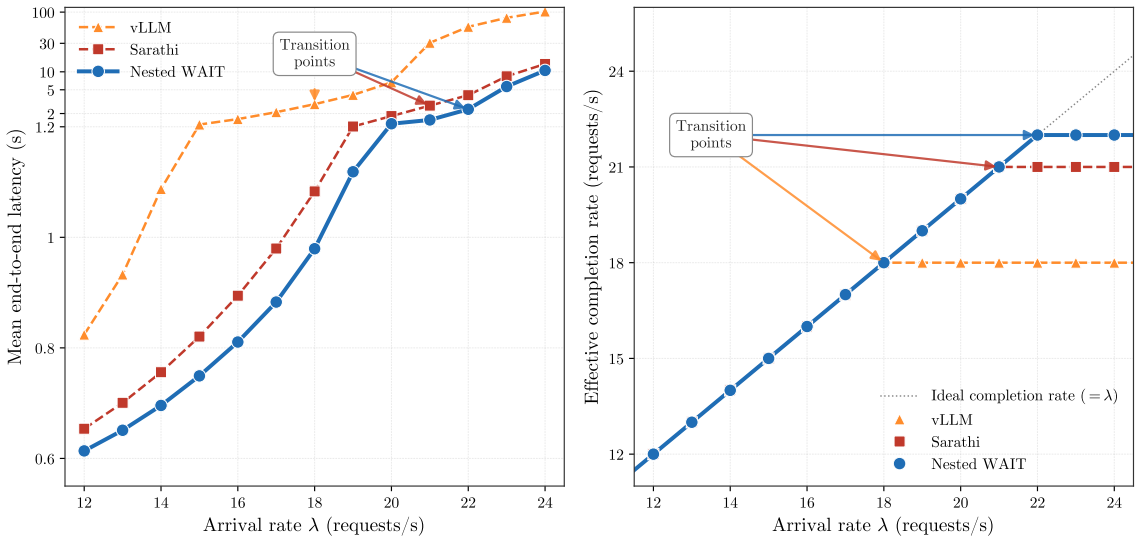

Figure 8 Two-type workload (0.7 `p512d20`, 0.3 `p512d50`) (Poisson arrivals): mean end-to-end latency (left) and effective completion rate (right) versus λ. The stability boundaries vLLM ($\lambda^* = 18$) < Sarathi ($\lambda^* = 21$) < Nested WAIT ($\lambda^* = 22$) are visible in both panels.

Long-decode regime. We also study the long-decode workload `p512d1000` with Poisson arrivals at $\lambda = 0.5, 1.0, \ldots, 5.0$. Relative to the single-type workload `p512d20`, each request now occupies about 30× more KV cache, so all three policies lose stability at much smaller arrival rates. Figure 9 shows that at low rates $\lambda \leq 2.5$ the three policies are nearly indistinguishable, but near the boundary WAIT again has the lowest latency: at $\lambda = 4.5$ it attains 32.3 s versus 41.7 s for Sarathi and 53.8 s for vLLM. The intuition is that long-decode requests stay in the system for many more iterations, so excess admission persists longer and makes it harder to keep arrivals and completions in balance. WAIT performs better in this regime because it keeps the in-system inventory closer to that balance point. The same pattern is already visible at $\lambda = 4.0$, where the baseline-memory setting gives 26.4 s for WAIT versus 30.2 s for Sarathi. Table 1 reports the accompanying near-capacity eviction comparison at the same arrival rate: in the tightened-memory run, Sarathi incurs a 2.88% eviction-induced restart rate, whereas WAIT still avoids eviction altogether. Under the server's last-in-first-out eviction rule, each evicted request loses its partial decode progress and begins again from prefill, so the restart gap is another manifestation of the same imbalance.

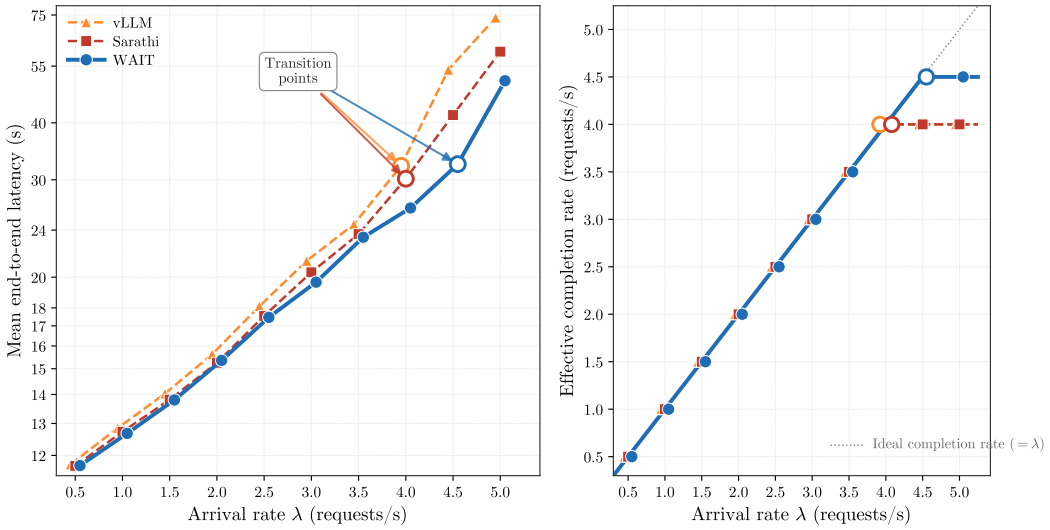

Figure 9 Long-decode workload (`p512d1000`, prefill 512, decode 1000): mean end-to-end latency (left) and effective completion rate (right) versus arrival rate λ. At low rates the three policies are largely comparable, with only small mixed latency differences; the main separation emerges from $\lambda \approx 3.0$ onward. Near the boundary, WAIT attains the lowest latency and sustains the highest completion rate.

Long-horizon validation of the stability boundary. To validate the estimated boundary, we vary the simulation horizon at the two arrival rates nearest Nested WAIT's transition. Figure 10 plots mean latency on $(0.7\,\texttt{p512d20}, 0.3\,\texttt{p512d50})$ for $\lambda \in \{22, 23\}$ as N increases from 2,000 to

Metric	Sarathi	WAIT (ours)
Mean end-to-end latency ($\lambda = 4.0$)	30.2 s	26.4 s
Eviction-induced restart rate ($\lambda = 4.0$)	2.88%	0%

Table 1 Near-capacity eviction diagnostic for the long-decode workload `p512d1000` at $\lambda = 4.0$. Under tightened memory, Sarathi incurs nonzero eviction-induced restart while WAIT avoids eviction altogether; the same comparison also shows lower latency under WAIT.

20,000. At $\lambda = 22$, Nested WAIT approaches a finite latency plateau, whereas Sarathi's mean latency grows approximately linearly in N, consistent with instability. At $\lambda = 23$, both policies are unstable, but Nested WAIT remains 30%–50% below Sarathi at every horizon we consider. Taken together, these comparisons are consistent with Nested WAIT being stable at $\lambda = 22$ and unstable at $\lambda = 23$, so the transition seen in the main figures persists as the horizon lengthens.

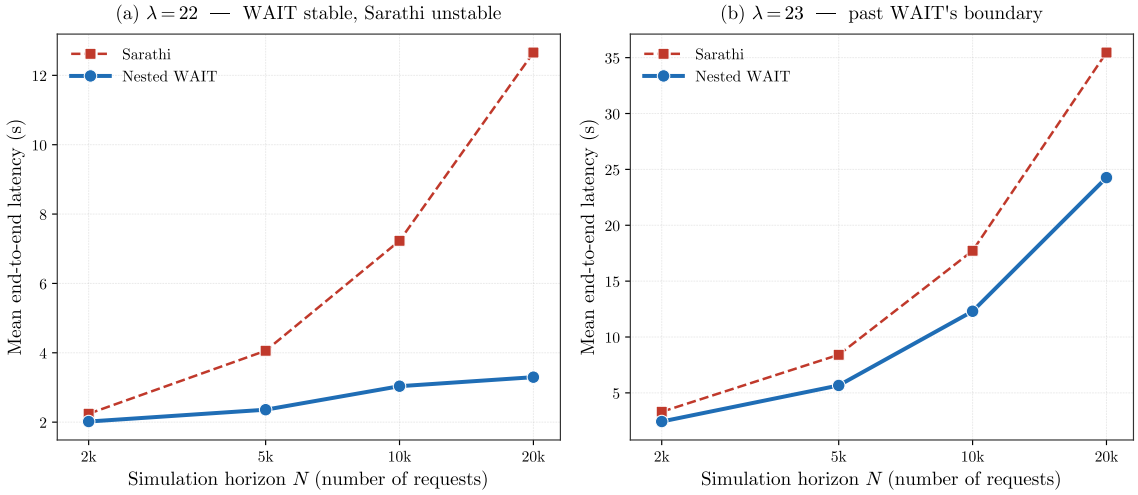

Figure 10 Long-horizon validation on the two-type workload (0.7 `p512d20`, 0.3 `p512d50`). **(a)** At $\lambda = 22$, Nested WAIT approaches a finite latency plateau, whereas Sarathi's mean latency grows roughly linearly in N. **(b)** At $\lambda = 23$, both policies are unstable, but Nested WAIT remains substantially below Sarathi at every horizon we consider.

6.2. Application to the `lmsys-chat-1m` Dataset

We next evaluate the policies on `lmsys-chat-1m`, a real dataset with heterogeneous prompt and response lengths, where decode lengths are unknown at admission. This empirical setting lets us assess the robustness of Nested WAIT's state-dependent admission rule under a heterogeneous workload distribution, and characterize how the induced capacity allocation varies across decode stages.

Dataset. The dataset contains approximately 1 million human–LLM conversations; we treat each user message as the prompt (prefill) and the corresponding assistant response as the generated output (decode). Figure 11 shows the resulting prefill-length and decode-length distributions. For simulation, we sample 5,000 requests from this dataset. In the sampled dataset, prefill lengths are concentrated near 35 tokens (median 30, 95th percentile 120), while decode lengths range from 1 to 500 tokens (median 95, 95th percentile 380). Short responses remain common: about 22% of prompts decode for at most 50 tokens, whereas about 10% exceed 300. Arrivals follow a Poisson process with rate λ, measured in requests per second. All runs use the same simulator, hardware, and baseline settings as in Section 6.1.

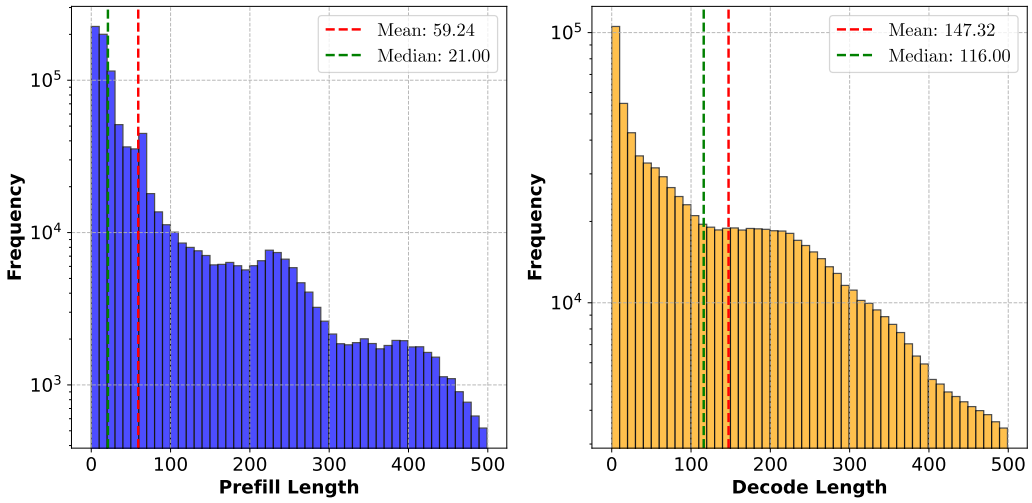

Figure 11 Raw prompt-length distributions in `lmsys-chat-1m`: prefill lengths on the left and decode lengths on the right. The dashed lines mark the mean and median of each distribution.

Segment construction and threshold allocation for Nested WAIT. Nested WAIT (Algorithm 2) regulates admission by partitioning the decode horizon into ordered segments, each with its own threshold. A request is counted against the threshold of its current segment: it first consumes capacity in the head segment and enters downstream segments only if it continues decoding. The policy therefore allocates admission capacity across realized continuation stages, rather than making admission decisions from predicted final response lengths.

Nested WAIT is distribution-aware: its thresholds are constructed from the empirical workload distribution and the offered arrival rate. For a chosen number of segments L, the decode horizon 0–500 is divided into L equal-width segments, and a request belongs to the segment containing

its current decode stage. Let $\Delta l'_k$ be the number of decode stages in segment k, and let λ'_k be the arrival rate of requests whose final decode length lies in that segment. A request reaches segment k only if its decode length exceeds the previous segment boundary, so the continuation probability from segment k to segment $k+1$ is $p_k = (\sum_{r=k+1}^{L} \lambda'_r)/(\sum_{r=k}^{L} \lambda'_r)$. Given a margin parameter η (the implementation parameter `seg_margin`), we set $q_k = \min\{p_k + \eta, 0.99\}$, making the downstream threshold ratios slightly larger than the empirical continuation probabilities. The real-valued per-stage thresholds are $n_k = n_1 \prod_{r<k} q_r$, where n_1 is chosen to satisfy $\sum_{k=1}^{L} \Delta l'_k n_k = \text{tl}$. Thus, the system-wide batch-size cap tl fixes the overall scale, while L and the continuation probabilities determine how that scale is allocated across decode segments. Before scheduling, each segment-level cap $B_k = \Delta l'_k n_k$ is rounded to an integer and spread as evenly as possible across the $\Delta l'_k$ stages in that segment. In the real-data runs, we set $\eta = 0.05$, choose tl from $40, 60, \ldots, 300$, and choose L from $\{1, 2, 3, 4, 5, 10, 20\}$ for each plotted arrival rate. Larger workloads use larger caps and finer segmentations to regulate downstream decode congestion at a finer resolution; lighter workloads use smaller caps and coarser segmentations so that the policy does not hold work unnecessarily.

Results. Figure 12 varies the arrival rate λ from 10 to 150 in steps of 10. The latency curves show the same qualitative ordering as in the synthetic experiments. The baseline schedulers are competitive when the workload is light, but their latencies increase more quickly as the system approaches and then exceeds the range in which completions can keep pace with arrivals. Nested WAIT has lower latency relative to the baselines, with the advantage becoming more significant in the near-overloaded and overloaded regimes. This is because long responses create downstream decode buildup: many requests finish in the early decode stages, while a smaller group remains active for hundreds of generated tokens. Nested WAIT's nested thresholds control how many requests may continue across successive decode-stage ranges, allowing short responses to clear early while limiting the later-stage population that consumes KV-cache capacity.

7. Conclusion and Future Directions

This paper studies online scheduling for LLM inference, a setting in which each request's memory consumption grows during decoding and exceeding capacity triggers evictions that can destabilize a system whose capacity is theoretically sufficient for stability. We develop a multi-stage online scheduling model that captures these dynamics and analyze a fluid approximation whose ideal-fluid equilibrium identifies both a stability benchmark and the memory requirement M^* needed to support it. Guided by this equilibrium, we design the (Nested) WAIT algorithm, a threshold-based

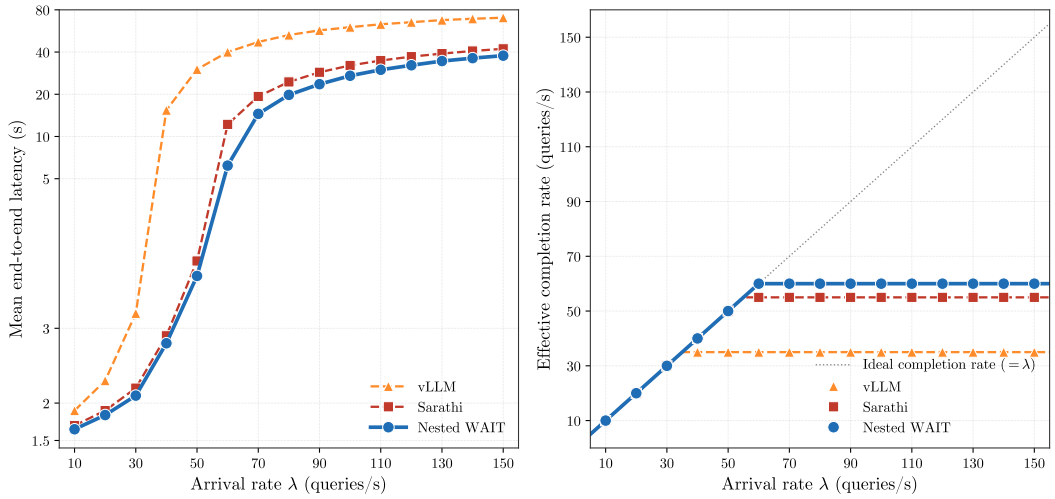

Figure 12 Real-data experiment on `lmsys-chat-1m` (5,000 requests): mean end-to-end latency (left) and effective completion rate (right) as the arrival rate λ varies. Nested WAIT has lower latency than the baseline schedulers, with the gap becoming more significant in the near-overloaded and overloaded regimes.

admission rule that keeps the stochastic system close to the ideal-fluid allocation across decode stages. The threshold mechanism performs a dimensionality reduction: it decouples interactions across prompt types and replaces the continuously growing KV cache dynamics with a discrete-time per-type recursion. Under this reduction, we establish asymptotic optimality for the effective throughput, the completion rate net of eviction, even when output lengths are unknown at arrival. Because the algorithms operate near the ideal-fluid equilibrium and prevent the eviction cascades that destabilize memory-reactive baselines, they both enlarge the realized stability region and reduce mean end-to-end latency across the underloaded, near-capacity, and overloaded regimes. Numerical experiments on Llama-2-7B with an NVIDIA A100 GPU confirm both effects relative to vLLM and Sarathi.

Several questions remain open. First, our threshold design relies on knowledge of the prompt-type arrival distribution; developing distribution-free variants, or versions that adapt online under bounded misspecification, would extend the algorithm's applicability. Second, practical deployments face arrival rates that drift over time and occasionally surge. Theorem 3 handles bounded rate variation by periodically re-solving the ideal-fluid equilibrium and its memory requirement, but sharper volatility, such as demand spikes that temporarily exceed nominal capacity, calls for further analysis; two concrete questions are how fast the thresholds can re-tune while preserving asymptotic optimality, and how transient surges interact with the safety buffer in Nested WAIT.

Finally, extending the analysis to multi-GPU deployments introduces inter-device communication cost and a choice of parallelization strategy (data, tensor, or pipeline). This regime is especially relevant for mixture-of-experts (MoE) architectures (Shazeer et al. 2017, Fedus et al. 2022, Jiang et al. 2024), in which token-level routing distributes experts across GPUs and creates skewed memory and compute load that our single-GPU analysis does not capture. A unified framework that models inter-device communication inside the iteration-time model, and coordinates admission thresholds across GPUs in response to the routing distribution, is a natural open direction.

Endnotes

References

Agrawal A, Kedia N, Mohan J, Panwar A, Kwatra N, Gulavani B, Ramjee R, Tumanov A (2024a) Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems* 6:351–366.

Agrawal A, Kedia N, Panwar A, Mohan J, Kwatra N, Gulavani B, Tumanov A, Ramjee R (2024b) Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 117–134.

Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani BS, Ramjee R (2023) Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* .

Aminabadi RY, Rajbhandari S, Awan AA, Li C, Li D, Zheng E, Ruwase O, Smith S, Zhang M, Rasley J, He Y (2022) Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15 (IEEE).

Anthropic (2023) Claude. https://claude.ai.

Anthropic (2025) Claude 3.7 sonnet. URL https://www.anthropic.com/claude/sonnet.

Asmussen S (2003) *Applied probability and queues*, volume 2 (Springer).

Bai J, Bai S, Chu Y, Cui Z, Dang K, Deng X, Fan Y, Ge W, Han Y, Huang F, et al. (2023) Qwen technical report. *arXiv preprint arXiv:2309.16609* .

Balkanski E, Rubinstein A, Singer Y (2016) The power of optimization from samples. *Advances in Neural Information Processing Systems* 29.

Bäuerle N (2002) Optimal control of queueing networks: An approach via fluid models. *Advances in Applied Probability* 34(2):313–328.

Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, et al. (2020) Language models are few-shot learners. *Advances in neural information processing systems* 33:1877–1901.

Chen Z, Ye Y, Zhou Z (2025) Adaptively robust llm inference optimization under prediction uncertainty. *arXiv preprint arXiv:2508.14544* .

Cole R, Roughgarden T (2014) The sample complexity of revenue maximization. *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, 243–252.

DeepSeek-AI (2024) Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. URL https://arxiv.org/abs/2405.04434.

DeepSeek-AI (2025) Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. URL https://arxiv.org/abs/2501.12948.

Desislavov R, Martínez-Fernández S, Franch X (2021) Compute and energy consumption trends in deep learning inference. *arXiv preprint arXiv:2109.02132* .

Devanur NR, Hayes TP (2009) The adwords problem: online keyword matching with budgeted bidders under random permutations. *Proceedings of the 10th ACM conference on Electronic commerce*, 71–78.

Devlin J, Chang MW, Lee K, Toutanova K (2019) Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 4171–4186.

Durrett R (2019) *Probability: theory and examples*, volume 49 (Cambridge university press).

Fedus W, Zoph B, Shazeer N (2022) Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23(120):1–39.

Fu Y, Zhu S, Su R, Qiao A, Stoica I, Zhang H (2025) Efficient llm scheduling by learning to rank. *Advances in Neural Information Processing Systems* 37:59006–59029.

García-Martín E, Rodrigues CF, Riley G, Grahn H (2019) Estimation of energy consumption in machine learning. *Journal of Parallel and Distributed Computing* 134:75–88.

GitHub (2022) Copilot. URL https://github.com/features/copilot.

Google (2023) Bard. https://bard.google.com.

Hooper C, Kim S, Mohammadzadeh H, Mahoney MW, Shao S, Keutzer K, Gholami A (2025) Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37:1270–1303.

Im S, Moseley B (2013) Online batch scheduling for flow objectives. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 102–104.

Jaillet P, Jiang J, Mellou K, Molinaro M, Podimata C, Zhou Z (2025) Online scheduling for llm inference with kv cache constraints. URL https://arxiv.org/abs/2502.07115.

Jiang AQ, Sablayrolles A, Roux A, Mensch A, Savary B, Bamford C, Chaplot DS, Casas Ddl, Hanna EB, Bressand F, et al. (2024) Mixtral of experts. URL https://arxiv.org/abs/2401.04088.

Kang H, Zhang Q, Kundu S, Jeong G, Liu Z, Krishna T, Zhao T (2024) Gear: An efficient kv cache compression recipefor near-lossless generative inference of llm. *arXiv e-prints* arXiv–2403.

Kaplan J, McCandlish S, Henighan T, Brown TB, Chess B, Child R, Gray S, Radford A, Wu J, Amodei D (2020) Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361* .

Kingman JFC (1962) On queues in heavy traffic. *Journal of the Royal Statistical Society: Series B (Methodological)* 24(2):383–392.

Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez J, Zhang H, Stoica I (2023) Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.

Lattanzi S, Lavastida T, Moseley B, Vassilvitskii S (2020) Online scheduling via learned weights. *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1859–1877 (SIAM).

Li R, Pal S, Pullu VN, Ryoo JH, John LK, Yadwadkar NJ (2025a) Oneiros: Kv cache optimization through parameter remapping for multi-tenant llm serving. *arXiv preprint arXiv:2507.11507* To appear in ACM SoCC 2025.

Li W, Wang L, Chai X, Yuan H (2020) Online batch scheduling of simple linear deteriorating jobs with incompatible families. *Mathematics* 8(2):170.

Li Y, Dai J, Peng T (2025b) Throughput-optimal scheduling algorithms for llm inference and ai agents. URL https://arxiv.org/abs/2504.07347, accessed: 2025-04-11.

Liu P, Lu X (2015) Online unbounded batch scheduling on parallel machines with delivery times. *Journal of Combinatorial Optimization* 29:228–236.

Liu Y, Whitt W (2011) A network of time-varying many-server fluid queues with customer abandonment. *Operations research* 59(4):835–846.

Lucier B, Menache I, Naor J, Yaniv J (2013) Efficient online scheduling for deadline-sensitive jobs. *Proceedings of the twenty-fifth annual ACM symposium on Parallelism in algorithms and architectures*, 305–314.

Maglaras C (2000) Discrete-review policies for scheduling stochastic networks: Trajectory tracking and fluid-scale asymptotic optimality. *The Annals of Applied Probability* 10(3):897–929.

Mandelbaum A, Massey WA, Reiman MI (1998) Strong approximations for markovian service networks. *Queueing Systems* 30(1):149–201.

Microsoft (2023) Bing ai. https://www.bing.com/chat.

OpenAI (2024) Gpt-4 technical report. URL https://arxiv.org/abs/2303.08774.

Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A, et al. (2022) Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35:27730–27744.

Patel D (2023) Peeling The Onion's Layers: Large Language Models' Cost, Context, and Feature Breakdown. Semianalysis blog, URL https://semianalysis.com/2023/02/13/peeling-the-onions-layers-large-language/, accessed: 2026-04-28.

Patel P, Choukse E, Zhang C, Shah A, Goiri Í, Maleki S, Bianchini R (2024) Splitwise: Efficient generative llm inference using phase splitting. in 2024 acm/ieee 51st annual international symposium on computer architecture (isca). *IEEE Computer Society, Los Alamitos, CA, USA* 118–132.

Patterson D, Gonzalez J, Le Q, Liang C, Munguia LM, Rothchild D, So D, Texier M, Dean J (2021) Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350* .

Pope R, Douglas S, Chowdhery A, Devlin J, Bradbury J, Heek J, Xiao K, Agrawal S, Dean J (2023) Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5:606–624.

SGLang Team (2024) Sglang: Fast serving framework for large language models and vision language models. https://github.com/sgl-project/sglang, gitHub repository.

Shazeer N, Mirhoseini A, Maziarz K, Davis A, Le Q, Hinton G, Dean J (2017) Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*.

Sheng Y, Zheng L, Yuan B, Li Z, Ryabinin M, Chen B, Liang P, Ré C, Stoica I, Zhang C (2023) Flexgen: High-throughput generative inference of large language models with a single gpu. *International Conference on Machine Learning*, 31094–31116 (PMLR).

Strait P (1974) On the maximum and minimum of partial sums of random variables. *Pacific Journal of Mathematics* 52(2):585–593.

Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243* .

Tay Y, Dehghani M, Bahri D, Metzler D (2022) Efficient transformers: A survey. *ACM Computing Surveys* 55(6):1–28.

Touvron H, Lavril T, Izacard G, Martinet X, Lachaux MA, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, et al. (2023) Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* .

Vee E, Vassilvitskii S, Shanmugasundaram J (2010) Optimal online assignment with forecasts. *Proceedings of the 11th ACM conference on Electronic commerce*, 109–118.

vLLM Team (2025) vllm v1: A major upgrade to vllm's core architecture. https://docs.vllm.ai/en/latest/getting_started/v1_user_guide.html, blog post, January 27, 2025. "In vLLM V1, the default preemption mode is RECOMPUTE rather than SWAP, as recomputation has lower overhead in the V1 architecture".

Wang M, Ye Y, Zhou Z (2025) Llm serving optimization with variable prefill and decode lengths. *arXiv preprint arXiv:2508.06133* .

Wei J, Tay Y, Bommasani R, Raffel C, Zoph B, Borgeaud S, Yogatama D, Bosma M, Zhou D, Metzler D, et al. (2022a) Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* .

Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D, et al. (2022b) Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35:24824–24837.

Wu CJ, Raghavendra R, Gupta U, Acun B, Ardalani N, Maeng K, Chang G, Aga F, Huang J, Bai C, et al. (2022) Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of Machine Learning and Systems* 4:795–813.

Yu GI, Jeong JS, Kim GW, Kim S, Chun BG (2022) Orca: A distributed serving system for {Transformer-Based} generative models. *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–538.

Zheng L, Chiang WL, Sheng Y, Li T, Zhuang S, Wu Z, Zhuang Y, Li Z, Lin Z, Xing EP, et al. (2023) Lmsys-chat-1m: A large-scale real-world llm conversation dataset. *arXiv preprint arXiv:2309.11998* .

Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H (2024) {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 193–210.

Appendix A: Extensions

The WAIT and Nested WAIT algorithms assume constant arrival rates and a fixed number of segments matching the number of prompt types. In practice, however, LLM inference systems face two additional challenges: (i) **time-varying workloads**, where arrival rates fluctuate throughout the day (e.g., peak hours vs. off-hours); (ii) **numerous prompt types**, where maintaining separate thresholds for hundreds of output lengths becomes infeasible. We address these challenges through two extensions that preserve the core algorithmic principles—eviction prevention and approaching load balance—while adapting to realistic deployment scenarios.

First, we extend Nested WAIT to handle time-varying arrival rates λ_j^t by replacing constant-rate threshold conditions with accumulated arrival constraints over each iteration window. The coupling-based proof technique extends naturally, requiring only that threshold conditions hold at every time instant (Section A.1). Second, we introduce a **segment-wise design** that groups many prompt types into a smaller number of decode-stage segments, reducing algorithmic complexity while maintaining the same threshold structure (Section A.2).

A.1. Time-Varying Arrival Rates

Time-varying arrival rates capture daily patterns in real LLM services: morning traffic surges, afternoon lulls, evening peaks. The threshold mechanism remains valid as long as the accumulated arrivals during each iteration window stay below the completion capacity. Formally, we replace constant rates λ_j with time-dependent rates λ_j^t, assumed continuous and uniformly bounded by $\lambda_{\max} < \infty$ to prevent unbounded bursts.

We analyze the asymptotic optimality of the Nested WAIT algorithm under time-varying arrival rates, denoted λ_j^t for prompt type $j \in [m]$ at time $t \in [0, T]$.

We define the accumulated arrivals of type j over the interval $[t_1, t_2]$ as $\lambda_j[t_1, t_2] := \int_{t_1}^{t_2} \lambda_j^t \, dt$. Recall from Equation (11) that $\Delta T_{[1,2,\ldots,m]}(n_1, \cdots, n_m)$ represents the per-iteration time cost when processing exactly n_k prompts at each stage in segment k, for all $k \in [m]$, as in (12), where n_k is the threshold defined in Algorithm 2.

For the Nested WAIT policy defined by $\pi = [n_1, \ldots, n_m]$, we consider the following conditions for all $t \in [0, T]$, $0 < \Delta t \le \Delta T_{[1,2,\cdots,m]}(n_1, \cdots, n_m)$ and $k = 1, 2, \ldots, m-1$, analogous to (12):

$$\begin{aligned} &\sum_{j=1}^{m} \lambda_j \left[t, t+\Delta t\right] \\ &\le \sum_{j=1}^{m} \lambda_j \left[t, t+\Delta T_{[1,2,\cdots,m]}(n_1, \cdots, n_m)\right] < n_1, \\ &n_k / n_{k-1} > p_k[t, t+\Delta t], \quad k = 2, \ldots, m, \end{aligned} \tag{15}$$

where $p_k[t, t+\Delta t] = (\sum_{j=k}^{m} \lambda_j[t, t+\Delta t]) / (\sum_{j=k-1}^{m} \lambda_j[t, t+\Delta t])$ for $k \ge 2$.

The following theorem extends Theorem 2 to time-varying arrival rates.

THEOREM 3. *Suppose the thresholds $\pi = [n_1, \ldots, n_m]$ satisfy (15), and the memory $M^{(\zeta,\pi)}$ fulfills:*

$$\begin{aligned} M^{(\zeta,\pi)} &\ge M^{\pi} + \sum_{k=2}^{m} (l + l'_{k-1}) \left(n_k + \theta_k^{-1} \ln\left(\tfrac{m\zeta T}{\delta(d_0 + d_1 M^{\pi})} \right) \right) \\ &= O\left(2M^{\pi} + \sum_{k=2}^{m} \theta_k^{-1} (l + l'_{k-1}) \ln\left(\tfrac{m\zeta T}{\delta(d_0 + d_1 M^{\pi})} \right) \right), \end{aligned}$$

where the memory usage M^{π} is defined as (13) and θ_k is given in (21) with the lower bound in Equation (22). The following asymptotic bounds hold:

$$\begin{aligned} &\boldsymbol{Throughput}^{*} - \mathbb{E}\left[\boldsymbol{Throughput}^{(\zeta,\pi)}\right] = O\left((\zeta T)^{-1}\right), \\ &\mathbb{E}\left[\boldsymbol{Latency}^{(\zeta,\pi)}\right], \mathbb{E}\left[\boldsymbol{TTFT}^{(\zeta,\pi)}\right] = O(1). \end{aligned}$$

Additionally, the memory usage remains below $M^{\zeta,\pi}$ with probability at least $1 - \delta$ over the time horizon $[0, T]$.

The extension to time-varying rates preserves the dimensionality reduction structure from Theorem 2. The key adaptation is that threshold conditions (15) must hold at every time instant $t \in [0, T]$, replacing constant rates λ_j with accumulated arrivals $\lambda_j[t, t+\Delta t]$ over each iteration window. Because arrival rates are continuous and bounded, the coupling construction remains valid: prompts transitioning from segment $k-1$ to segment k are controlled by the corresponding time-varying thinning probability. The Lindley recursion and Doob's maximal inequality apply identically, bounding queue lengths with the same logarithmic overhead. Appendix F provides the detailed proof.

A.2. Segment Design

When LLM systems handle many distinct output lengths, maintaining individual thresholds for each type becomes impractical: the algorithm must track queue lengths separately for every output length, increasing memory overhead and implementation complexity. The **segment-wise design** addresses this by grouping similar output lengths into $L \le m$ segments, where each segment k aggregates types with comparable decode lengths. This reduces algorithmic complexity from $O(m)$ to $O(L)$ while preserving the threshold logic: prompts still classify themselves on the fly, but now at segment boundaries rather than individual type boundaries.

We assume constant arrival rates for clarity, though the approach extends to time-varying rates as discussed in Section A.1. Consider m prompt types with decode lengths $l'_1 < \cdots < l'_m$ and arrival rates $\lambda_1, \lambda_2, \cdots, \lambda_m$. Our objective is to merge these into L segments, where $L \le m$. For notational convenience, let $\Delta L := m/L$. We partition the m types into L segments as follows: types 1 to ΔL, types $\Delta L + 1$ to $2\Delta L$, ..., types $(L-1)\Delta L + 1$ to m.

For each segment $k \in \{1, \ldots, L\}$, the total arrival rate is defined as $\lambda'_k = \sum_{(k-1)\Delta L+1 \le j \le k\Delta L} \lambda_j$. Additionally, we define $\lambda'(k_1 \to k_2) := \sum_{r=k_1}^{k_2} \lambda'_r$ for $1 \le k_1 \le k_2 \le L$.

Given a policy $\pi = [n_1, \ldots, n_L]$ with threshold n_k for the k-th segment in Algorithm 2, we formulate a linear system analogous to Equation (12):

$$\begin{aligned} &\Delta T_{[1:L]}(n_{1:L}) \le \frac{n_1}{\sum_{r=1}^{L} \lambda'_r}, \\ &\frac{n_k}{n_{k-1}} > p_k, \ \forall k = 2, \ldots, L, \end{aligned} \tag{16}$$

where $p_k = \frac{\lambda'(k \to L)}{\lambda'(k-1 \to L)}$ for $k \ge 2$. Let $r_k = l'_{k\Delta L}$ be the decode boundary of segment k, with $r_0 = 0$. Define $\delta_1 = r_1 + 1$, $\bar{s}_1 = r_1/2$, and for $k \ge 2$, $\delta_k = r_k - r_{k-1}$ and $\bar{s}_k = (r_{k-1} + 1 + r_k)/2$. Then $\Delta T_{[1,\ldots,L]}(n_1, \ldots, n_L) = d_0 + d_1 M^\pi(n_1, \ldots, n_L)$, where the base memory used by threshold-sized segment batches is

$$M^\pi(n_1, \ldots, n_L) = \sum_{k=1}^{L} n_k (l + \bar{s}_k)\delta_k.$$

For heterogeneous prefill lengths, the segment logic is unchanged because boundaries are determined by decode progress. A sufficient memory condition replaces the common l in each segment by a segment-wise upper bound $\bar{l}_k = \max\{l_j : \text{type } j \text{ can enter segment } k\}$, or by the corresponding type-weighted term when the type mix within the segment is known.

The segment-wise design achieves two practical goals: (i) **robustness** to varying or uncertain type distributions—if actual output lengths differ slightly from predictions, grouping into segments absorbs the variation; and (ii) **simplicity** in deployment—the scheduler maintains thresholds for a small number of decode-stage segments rather than for every possible output length. The theoretical guarantee shows that this simplification preserves the same asymptotic throughput rate, with the same logarithmic safety-buffer structure.

THEOREM 4. *For any fixed number of segments L and thresholds $\pi = [n_1, \ldots, n_L]$ satisfying Equation (16), if the memory capacity $M^{(\zeta,\pi)}$ satisfies:*

$$\begin{aligned} M^{(\zeta,\pi)} &\geq M^{\pi} + \sum_{k=2}^{L} (l + r_{k-1}) n_k \\ &\quad + \sum_{k=2}^{L} (l + r_{k-1}) \theta_k^{-1} \ln\left(\frac{m\zeta T}{\delta(d_0 + d_1 M^{\pi})}\right) \\ &= O\left(2M^{\pi} + \sum_{k=2}^{L} (l + r_{k-1}) \theta_k^{-1} \ln\left(\frac{m\zeta T}{\delta(d_0 + d_1 M^{\pi})}\right)\right), \end{aligned} \tag{17}$$

where θ_k is given by Equation (20) with lower bound $8(n_k - n_{k-1}p_k)/n_{k-1}$ for $k \geq 2$. The performance of Nested WAIT satisfies:

$$\begin{aligned} \boldsymbol{Throughput}^{*} - \mathbb{E}\left[\boldsymbol{Throughput}^{(\zeta,\pi)}\right] &= O\left((\zeta T)^{-1}\right), \\ \mathbb{E}\left[\boldsymbol{Latency}^{(\zeta,\pi)}\right] &= O(1), \\ \mathbb{E}\left[\boldsymbol{TTFT}^{(\zeta,\pi)}\right] &= O(1). \end{aligned}$$

Moreover, memory is not exceeded with probability at least $1 - \delta$ over the time horizon $t \in [0, T]$.

Clustering m types into L segments preserves the core threshold structure from Theorem 2. The key observation is that aggregation commutes with the coupling argument: individual arrival rates $(\lambda_1, \ldots, \lambda_m)$ are replaced by aggregated rates $(\lambda'_1, \ldots, \lambda'_L)$, where $\lambda'_k = \sum_{j \in \text{segment } k} \lambda_j$, and downstream arrivals are governed by the same binomial thinning argument at the segment boundaries. The Lindley recursion and Doob's maximal inequality apply identically at the segment level, yielding the same logarithmic overhead in Equation (17). The proof follows the structure of Theorem 2 in Appendix E.

We validate the segment-wise design numerically using a real-data workload derived from the LMSYS dataset Zheng et al. (2023) (Section 6). Decode lengths range up to 500 tokens in this

workload, and the distribution of arrival rates, shown in Figure 13, is proportional to normalized frequencies from real user traffic.

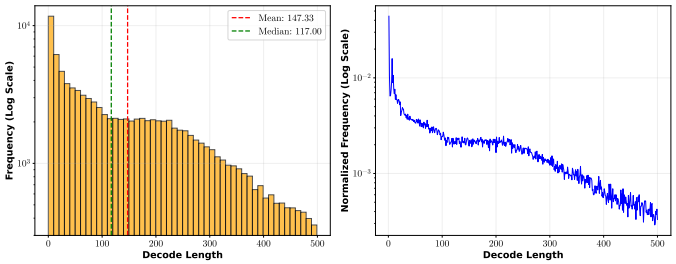

Figure 13 Arrival rate construction from real data, with rates proportional to normalized frequencies.

We analyze the three components of the memory bound in Equation (17): (1) the peak batch memory usage $M^{\pi} = \sum_{k=1}^{L} n_k(l + \bar{s}_k)\delta_k$, (2) the deterministic boundary-queue term $\sum_{k=2}^{L} n_k(l + r_{k-1})$, and (3) the high-probability boundary-queue term $\sum_{k=2}^{L} \theta_k^{-1}(l + r_{k-1}) \ln\left(\frac{(L-1)(T+1)}{\delta}\right)$.

Figures 14, 15, and 16 plot these three memory components (y-axis) as functions of the number of segments L (x-axis), under varying arrival rates, time horizons T, and confidence levels δ. Each panel shows: Term 1 (blue circles) representing the ideal-fluid memory requirement from batching, Term 2 (green squares) representing the deterministic queue bound, Term 3 (red triangles) representing the stochastic safety buffer with logarithmic dependence on T and δ, and their sum (black crosses) representing the total memory requirement $M^{(\zeta,\pi)}$. Three key observations emerge: (i) Term 1 (the ideal-fluid memory requirement) dominates Terms 2 and 3 across all parameter settings, confirming that the safety buffer remains modest relative to the base batching requirement. (ii) The total memory (black line) follows a **U-shaped curve** with respect to L: too few segments waste memory by over-provisioning for heterogeneous types (high Term 1 at small L), while too many segments increase overhead from maintaining separate queues. (iii) Term 3 exhibits smooth growth as δ decreases (comparing left vs. right panels in Figures 14 and 15) and as T increases (comparing Figure 16), consistent with the logarithmic bound in the theoretical analysis.

These extensions demonstrate that the core threshold-based design, eviction prevention through controlled admission, remains robust to realistic deployment conditions. Time-varying arrival rates and segment-wise grouping introduce no fundamental barrier; the coupling-based analysis extends naturally with logarithmic overhead. Section 6 illustrates this implementation on a real workload, where Nested WAIT uses coarser decode-stage segments rather than maintaining a separate threshold for every possible output length.

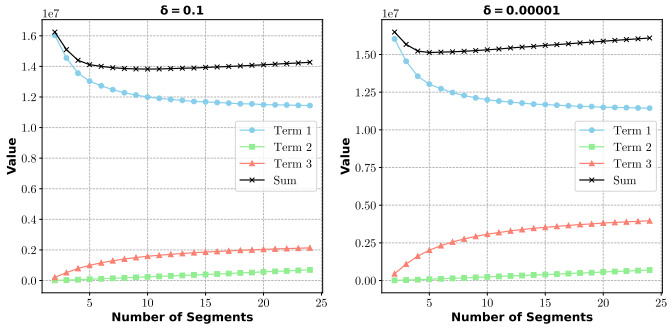

Figure 14 Memory usage proportions under low arrival rate (total rate = 50, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

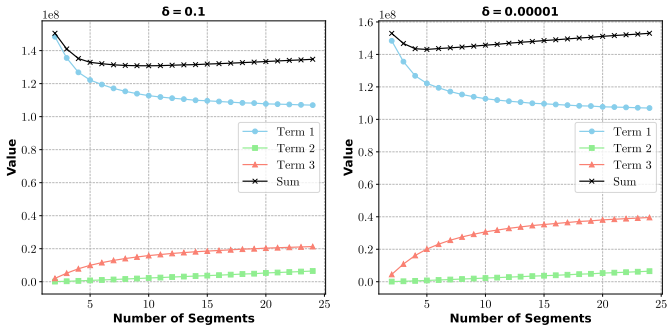

Figure 15 Memory usage proportions under high arrival rate (total rate = 500, $T = 200$). Left: $\delta = 0.1$; Right: $\delta = 10^{-5}$.

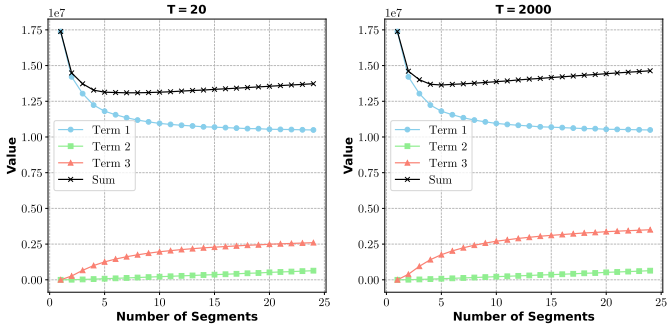

Figure 16 Memory usage proportions with varying time horizons (total rate = 50, $\delta = 0.001$). Left: $T = 20$; Right: $T = 2000$.

Appendix B: Notation Summary

Table 2 Summary of notation

Symbol	Description
m	Number of prompt types
j	Index of prompt type, $j \in \{1, \ldots, m\}$
λ_j	Arrival rate of type-j prompts
l_j	Input length of type-j prompts after prefill
l'_j	Output length of type-j prompts in the decode phase
s	Stage index: $s = 0$ for prefill and $s \in \{1, \ldots, l'_j\}$ for decode
k	Segment index in Nested WAIT
b	Batch index
n^t_{js}	Number of type-j prompts at stage s at time t
n_j	Per-stage threshold for type-j prompts in WAIT
n_k	Threshold for segment k in Nested WAIT
C	GPU memory capacity
$\mathcal{G}^t$	Set of prompts with KV cache on GPU at time t
B^t	Batch of prompts processed at iteration t, with $B^t \subseteq \mathcal{G}^t$
τ	Iteration time to process a batch
d_0	Fixed overhead time per batch iteration
d_1	Time cost per unit of KV cache memory
M^*	Memory requirement needed to support the ideal-fluid equilibrium
n^*_j	Equilibrium per-stage inventory of type-j prompts
$\mathbf{Throughput}^{(T,\pi)}$	Effective throughput of policy π over horizon $[0, T]$
$\mathbf{Latency}^{(T,\pi)}$	Average latency of policy π over horizon $[0, T]$
$\mathbf{TTFT}^{(T,\pi)}$	Average time to first token of policy π over horizon $[0, T]$
$\mathbf{Throughput}^*$	Equilibrium effective throughput, measured in completed decode tokens per unit time
Π	Class of admissible scheduling policies

Appendix C: Proofs of Propositions

C.1. Proof of Proposition 1

Proof Outline. We show that when $\lambda_j d_1 (l'_j + 1)(l_j + l'_j/2) \geq 1$, arrivals consistently exceed completions regardless of batch size, causing queue length to grow unboundedly.

C onsider a system with only type-j prompts and suppose q_j prompts are processed at each stage. The total memory consumption is:

$$M_j = q_j \sum_{s=0}^{l'_j} (l_j + s) = q_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right).$$

The iteration time to process this batch is:

$$\tau_j = d_0 + d_1 M_j = d_0 + d_1 q_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right).$$

During this iteration, the expected number of arriving prompts is:

$$\text{Arrivals}_j = \tau_j \cdot \lambda_j = d_0 \lambda_j + \lambda_j d_1 q_j (l'_j + 1) \left(l_j + \frac{l'_j}{2} \right),$$

while the number of completions is q_j (prompts at stage l'_j complete). The arrival-completion gap satisfies:

$$\text{Arrivals}_j - q_j = d_0 \lambda_j + q_j \left[\lambda_j d_1(l'_j + 1) \left(l_j + \frac{l'_j}{2} \right) - 1 \right] \geq d_0 \lambda_j > 0,$$

where the inequality follows from $\lambda_j d_1 (l'_j + 1)(l_j + l'_j/2) \geq 1$.

Since this gap is positive for all $q_j > 0$, the queue length after N iterations satisfies:

$$\text{Queue Length}(N) \geq d_0 \lambda_j \cdot N.$$

The latency for a prompt arriving at iteration N grows as $\Omega(N)$, establishing instability.

C.2. Proof of Proposition 2

Proof Outline. The expected throughput of any online policy cannot exceed the fluid throughput **Throughput***, since throughput is bounded by the arrival rate of output tokens.

I n the fluid model, the equilibrium parameters are:

$$\Delta T = \frac{d_0}{1 - d_1 \sum_{j=1}^{m} \lambda_j (l'_j + 1)(l_j + l'_j/2)}, \quad n_j^* = \Delta T \cdot \lambda_j,$$

$$M^* = \Delta T \cdot \sum_{j=1}^{m} \lambda_j (l'_j + 1)(l_j + l'_j/2), \quad \textbf{Throughput}^* = \sum_{j=1}^{m} \lambda_j l'_j.$$

In any stochastic system, throughput is counted only after prompts complete. Over time horizon $[0, T]$, let Arrivals_j^T denote the total arrivals of type-j prompts. The total output tokens generated cannot exceed the total output length of all arrivals:

$$\text{Throughput}^{(\pi,T)} \leq \sum_{j=1}^{m} \text{Arrivals}_j^T \cdot l'_j.$$

Taking expectations and dividing by T:

$$\mathbb{E}[\textbf{Throughput}^{(T,\pi)}] \leq \sum_{j=1}^{m} \mathbb{E}\left[\frac{\text{Arrivals}_j^T}{T}\right] \cdot l'_j = \sum_{j=1}^{m} \lambda_j l'_j = \textbf{Throughput}^*.$$

C.3. Proof of Proposition 3

Proof Outline. We construct a two-type example where FCFS suffers from cascading evictions even when capacity equals the ideal-fluid memory requirement M^*, leading to an $\Omega(1)$ throughput gap.

*C*onsider two prompt types: type 1 with $(l_1, l'_1, \lambda_1) = (1, 1, 1)$ and type 2 with $(l_2, l'_2, \lambda_2) = (1, 2, 1)$.

Fluid Equilibrium. Under the per-stage convention for n_j^*, the equilibrium equations are $d_0 + d_1(3n_1 + 6n_2) = n_1 = n_2$, which yield:

$$K = \frac{d_0}{1 - 9d_1}, \quad n_1^* = K, \quad n_2^* = K, \quad M^* = 9K.$$

Setting $K = 1$ (without loss of generality), we have $n_1^* = n_2^* = 1$ and $M^* = 9$.

Equilibrium Batch Composition. In equilibrium, the batch contains:

- Type 1: 1 prompt at stage 0 (memory = 1), 1 prompt at stage 1 (memory = 2, completing)
- Type 2: 1 prompt at stage 0 (memory = 1), 1 at stage 1 (memory = 2), 1 at stage 2 (memory = 3, completing)

Overflow Probability. Let (X_1, X_2) denote arrivals during one iteration, where $X_j \sim \text{Poisson}(1)$. The combinations that do not cause memory overflow within two iterations are: $(0,0), (1,0), (0,1), (1,1), (2,0)$. All other combinations lead to overflow. The overflow probability is:

$$p = 1 - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{e^2} - \frac{1}{2e^2} = 1 - \frac{9}{2e^2} \approx 0.391.$$

Since overflow probability is bounded away from zero, FCFS experiences eviction with constant probability at each iteration. This leads to queue length growth $\Omega(t)$ and throughput gap $\Omega(1)$.

C.4. Proof of Proposition 5

Proof Outline. Using the same two-type example, we show that when output lengths are unknown until completion, any online policy faces either memory underutilization or constant overflow probability.

C onsider the same setup as Proposition 3: type 1 with $(l_1, l_1', \lambda_1) = (1, 1, 1)$ and type 2 with $(l_2, l_2', \lambda_2) = (1, 2, 1)$. The ideal-fluid balance gives $M^* = 9$.

The key constraint is that prompt types cannot be distinguished until a type-2 prompt enters stage 2 (generates its second output token). At stage 0 and stage 1, both types appear identical to the scheduler.

Consider any admissible batching policy. For each possible batch configuration, one of the following must hold:

1. **Memory underutilization:** The batch leaves significant memory unused, causing expected arrivals to exceed completions during the iteration.

2. **Overflow risk:** The batch fully utilizes memory, but faces constant probability $\rho > 0$ of overflow when arrivals exceed expectations.

Since throughput is counted only upon prompt completion, and the fluid throughput equals expected arrivals' output length per unit time, any policy that either underutilizes memory or experiences overflow incurs a throughput gap of at least $\Omega(\rho)$ compared to **Throughput***.

Appendix D: Proof of Theorem 1

Proof Outline. This proof establishes asymptotic optimality of the WAIT algorithm when output lengths are known. The structure proceeds in two parts:

1. **Single-type case**: We construct a coupled dominating process using the Lindley recursion and apply Kingman's bound for queue length expectations.

2. **Multiple-type case**: We introduce a conservative periodic comparison policy whose slots have the full-threshold processing time. This separates the random batch composition of event-driven WAIT from the deterministic clock used in the drift argument.

Key techniques include: Lindley recursion for queue length bounds (Asmussen 2003, Proposition 6.3), coupling construction to dominate the original process, Kingman's bound for negative drift processes (Kingman 1962), and standard random walk results (Strait 1974).

Throughout this appendix, we use b to denote the embedded comparison-slot index, which is distinct from the stage index $s \in \{0, 1, \ldots, l'_j\}$ used in the main text for decode stages. We use t for continuous time and B for the number of comparison slots.

We first consider the single-type case, then extend to multiple types.

D.0.1. Single-Type Case Consider a single prompt type with threshold n. For general scheduling algorithms, analyzing this system in continuous time is challenging: KV cache grows dynamically during decode, arrivals are stochastic, and eviction risks complicate the dynamics.

The WAIT policy's *threshold mechanism achieves a dimensionality reduction*. By waiting until exactly n prompts accumulate before initiating a batch, the algorithm enforces a fixed batch size, which ensures constant processing time ΔT per batch. This decouples the continuous-time memory-constrained dynamics into a tractable discrete-time process: a memory-constrained queueing system becomes a simple random walk. The reduction is enabled by the algorithm design and is what makes the subsequent drift analysis apply.

Specifically, let $\tilde{\lambda}$ denote the continuous-time arrival rate. The WAIT policy's fixed batch size implies that arrivals during each batch follow Poisson$(\tilde{\lambda} \cdot \Delta T)$. Defining $\lambda = \tilde{\lambda} \cdot \Delta T$ (expected arrivals per batch), we can analyze the system as an *embedded Markov chain* observed at batch completion times, indexed by $b \in \mathbb{N}$. Setting $n = \lambda$ corresponds to critical loading. The throughput loss arises from "stuck" iterations where insufficient prompts are available.

LEMMA 1 **(Queue Length and Stuck Time)**. *This lemma connects queue accumulation to throughput loss through stuck iterations. Let X^b denote the number of arrivals during batch b, where $X^b \sim$ Poisson(λ). Let W^b denote the queue length after batch b, with $W^0 = 0$. The state transition is:*

$$W^{b+1} = W^b + X^b - \lambda \cdot \mathbf{1}\{W^b + X^b \geq \lambda\}.$$

Define B_{stuck} as the number of stuck iterations (where $W^b + X^b < \lambda$). Then:

$$\mathbb{E}[W^B] = \lambda \cdot \mathbb{E}[B_{\text{stuck}}].$$

The proof is in Appendix G.1. This lemma connects queue accumulation to throughput loss: stuck iterations directly translate to waiting prompts. To bound $\mathbb{E}[W^B]$, we construct a coupled dominating process that admits tractable analysis via classical queueing techniques.

LEMMA 2 **(Coupled Dominating Process).** *The coupled process starts with a safety buffer (2λ vs 0) and maintains dominance throughout. Define $\{\tilde{W}^b\}$ by:*

$$\tilde{W}^0 = 2\lambda, \quad \tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\}.$$

Then $\tilde{W}^b \geq W^b + \lambda$ for all $b \in \mathbb{N}$.

The proof is in Appendix G.2.

Lindley recursion. To analyze the coupled process, we apply the Lindley recursion representation (Asmussen 2003, Proposition 6.3):

LEMMA 3 **(Lindley Representation).** *Let $S^k = \sum_{r=1}^{k}(X^r - \lambda)$ be the partial sum process. Then:*

$$\tilde{W}^B = 2\lambda + \max(S^B, S^B - S^1, \ldots, S^B - S^{B-1}, 0).$$

This classical queueing representation is made applicable by the WAIT policy's threshold design. Define $\xi^k = X^k - \lambda$. We use time-reversal symmetry to simplify the maximum expression:

$$\max(S^B, S^B - S^1, \ldots, S^B - S^{B-1}, 0) = \max_{1\leq k\leq B}\left(\sum_{r=k}^{B}\xi^r\right)^+ \overset{d}{=} \max_{1\leq k\leq B}\left(\sum_{r=1}^{k}\eta^r\right)^+, \quad \eta^r \overset{d}{=} X^1 - \lambda.$$

Thus $\tilde{W}^B \overset{d}{=} 2\lambda + \max_{1\leq k\leq B}(\sum_{r=1}^{k}\eta^r)^+$.

Since $\tilde{W}^B \geq W^B + \lambda$, we have $\mathbb{E}[W^B] \leq \mathbb{E}[\tilde{W}^B] - \lambda$. Let $Y^B = \max_{1\leq k\leq B}(\sum_{r=1}^{k}\eta^r)^+$. By (Strait 1974, Lemma 2):

LEMMA 4 **(Random Walk Maximum).**

$$\mathbb{E}\left[Y^B\right] = \mathbb{E}\left[\max_{1\leq k\leq B}(S^k)^+\right] = \sum_{k=1}^{B}\frac{1}{k}\mathbb{E}\left[(S^k)^+\right].$$

This standard result (Strait 1974) decomposes the maximum into a sum over batch indices. For $\mathbb{E}[(S^k)^+]$, by the Cauchy-Schwarz inequality and $\mathrm{Var}(S^k) = \lambda k$, we have $\mathbb{E}[(S^k)^+] \leq \sqrt{\lambda k}$. Hence:

$$\mathbb{E}[Y^B] \leq \sqrt{\lambda}\sum_{k=1}^{B}k^{-1/2} \sim \sqrt{\lambda B} \quad \text{as } B \to \infty.$$

LEMMA 5 **(Throughput Gap Bound).**

$$\mathbb{E}[W^B] - \lambda \leq \mathbb{E}[Y^B] \leq \sqrt{\lambda}\sum_{k=1}^{B}\frac{1}{\sqrt{k}} \sim \sqrt{\lambda B}, \quad B \to \infty.$$

By Lemma 1, the expected number of stuck iterations is $\mathbb{E}[B_{stuck}] = \mathbb{E}[W^B]/\lambda = O(\sqrt{B})$, so the normalized throughput loss from stuck iterations is $O(B^{-1/2})$. The same bound applies to every prefix $b \le B$, and therefore

$$\frac{1}{B}\sum_{b=1}^{B}\mathbb{E}[W^b] \le \frac{1}{B}\sum_{b=1}^{B} O(\sqrt{b}) = O(\sqrt{B}).$$

This time-averaged backlog bound is the quantity used for the latency and TTFT estimates.

Scaling with the horizon. Let Δ^π denote the fixed processing time of a threshold-sized batch under policy π. In the scaled system, the corresponding processing time is Δ^π/ζ. Over a calendar horizon T, set

$$B_\zeta = \left\lfloor \frac{\zeta T}{\Delta^\pi} \right\rfloor.$$

Then $B_\zeta = \zeta T/\Delta^\pi + O(1)$. The terminal backlog bound gives normalized throughput loss $O(B_\zeta^{-1/2}) = O((\zeta T)^{-1/2})$. For delay, the theorem reports service-normalized time, so one scaled comparison slot has length Δ^π; the time-averaged backlog bound therefore gives latency and TTFT contributions $O(B_\zeta^{1/2}) = O((\zeta T)^{1/2})$. The bounded number of service stages is lower order. This yields the single-type throughput, latency, and TTFT orders stated in the theorem.

D.0.2. Multiple-Type Case We now extend to m known prompt types. The additional issue is that event-driven WAIT need not process the full type composition in every realized batch: if only a subset of type queues has reached its threshold, only that subset is batched. The proof therefore separates two objects. The actual policy is the event-driven WAIT policy from the main text. The comparison object is a periodic policy with deterministic full-threshold slots; it is slower than actual WAIT because it reviews less often and pads every selected batch to the full-slot length.

For a threshold vector $\pi = [n_1, \dots, n_m]$, let

$$\Delta^\pi \equiv \Delta T_{[1,\dots,m]} = d_0 + d_1 M^\pi, \qquad M^\pi = \sum_{j=1}^{m} n_j (l'_j + 1)\left(l_j + \frac{l'_j}{2}\right). \tag{18}$$

This is the processing time of the full threshold composition, i.e., the batch that contains n_j prompts from every decode stage of every type j. The load-balance and memory conditions are

$$\begin{aligned} &\lambda_j \Delta^\pi \le n_j \quad \text{for all } j \in [m], \\ &M^* \le M^\pi \le C. \end{aligned} \tag{19}$$

The first line says that, over one full-threshold iteration, the expected type-j arrivals do not exceed the n_j type-j prompts that can be advanced from each active stage. The second line is the corresponding

feasibility condition: the threshold composition is at least on the scale of the ideal-fluid memory requirement but does not exceed the physical memory C.

For any realized event-driven WAIT batch r, let $a_{js}^{(r)} \le n_j$ be the number of selected type-j prompts at decode stage s, and let J_r be the set of selected types. Its KV memory satisfies

$$M_r = \sum_{j\in J_r} \sum_{s=0}^{l'_j} a_{js}^{(r)} (l_j + s) \le \sum_{j=1}^{m} n_j \sum_{s=0}^{l'_j} (l_j + s) = M^\pi.$$

Therefore the physical processing time of the realized batch in the ζ-scaled system is

$$S_r^{(\zeta)} = \frac{d_0 + d_1 M_r}{\zeta} \le \frac{\Delta^\pi}{\zeta}.$$

This is the only direction needed for the comparison argument: actual batches may contain fewer types and have shorter duration, but no realized WAIT batch is larger than the full-threshold composition.

In the ζ-scaled system, one full-threshold slot has physical length Δ^π/ζ. Define deterministic review times

$$u_b = b\Delta^\pi/\zeta, \qquad b = 0, 1, 2, \ldots.$$

The comparison policy $\bar{\pi}$ reviews the entry queues only at these times. We use an end-of-slot accounting convention: arrivals accumulate over $(u_b, u_{b+1}]$, and at u_{b+1} the comparison policy removes n_j type-j prompts from each active stage whenever the type-j entry queue has reached n_j. Equivalently, one can view the selected work as occupying the following full slot; this one-slot shift changes only initial and terminal constants. Every selected batch is padded to length Δ^π/ζ, and the construction is feasible because any selected type subset has memory at most the full threshold composition $M^\pi \le C$.

The periodic comparison policy is deliberately conservative. Couple it with the event-driven WAIT policy on the same arrival sample path. Actual WAIT reviews at every arrival and batch completion, starts a batch as soon as a type threshold is met, and uses the realized batch composition rather than padding it to the full-threshold duration. Thus the work removed by $\bar{\pi}$ in any full comparison slot is work that actual WAIT could have started no later, unless actual WAIT was already processing earlier threshold work. Since every realized WAIT batch has processing time at most Δ^π/ζ, a monotonicity induction over $\{u_b\}$ gives that actual WAIT has no fewer completions

and no larger entry waiting contribution than $\bar{\pi}$, up to the bounded prompts in service and the final partial slot. It is therefore enough to bound the comparison policy; these bounded edge effects contribute $O((\zeta T)^{-1})$ to normalized throughput and $O(1)$ to the service-normalized delay metrics.

For the comparison policy, let $\bar{W}^b_{(j)}$ be the residual type-j entry queue after the b-th review, and let

$$\bar{X}^b_{(j)} = A_j(u_b, u_{b+1}] \sim \text{Poisson}\left(\zeta\lambda_j \frac{\Delta^\pi}{\zeta}\right) = \text{Poisson}(\lambda_j \Delta^\pi).$$

The arrivals $\{\bar{X}^b_{(j)}\}$ are independent across comparison slots and do not depend on other type queues. With the end-of-slot accounting convention above, the residual entry queue evolves as

$$\bar{W}^{b+1}_{(j)} = \bar{W}^b_{(j)} + \bar{X}^b_{(j)} - n_j \mathbf{1}\{\bar{W}^b_{(j)} + \bar{X}^b_{(j)} \geq n_j\}.$$

This is the point at which the batch-composition issue disappears: the only coupling across types is the deterministic feasibility condition $M^\pi \leq C$; under the comparison clock, each type has its own threshold queue.

LEMMA 6 (**Multiple-Type Comparison Queue Bound**). *For each type j, define*

$$\widetilde{W}^0_{(j)} = 2n_j, \qquad \widetilde{W}^{b+1}_{(j)} = \max\{2n_j, \widetilde{W}^b_{(j)} + \bar{X}^b_{(j)} - n_j\}.$$

Then $\widetilde{W}^b_{(j)} \geq \bar{W}^b_{(j)}$ for all $b \geq 0$ and $j \in [m]$.

The proof is in Appendix G.3. Let $\mu_j = \lambda_j \Delta^\pi$. The increment in the dominating process is $\bar{X}^b_{(j)} - n_j$, whose mean is $\mu_j - n_j \leq 0$ by (19). If $\mu_j = n_j$, the same random-walk maximum argument used in the single-type proof gives

$$\mathbb{E}[\bar{W}^B_{(j)}] \leq \mathbb{E}[\widetilde{W}^B_{(j)}] = O(B^{1/2}).$$

The same bound holds for every prefix $b \leq B$, so

$$\frac{1}{B}\sum_{b=0}^{B-1} \mathbb{E}[\bar{W}^b_{(j)}] = O(B^{1/2}).$$

If $\mu_j < n_j$, the dominating process has strictly negative drift. Since

$$\text{Var}(\bar{X}^b_{(j)} - n_j) = \mu_j, \qquad \left|\mathbb{E}[\bar{X}^b_{(j)} - n_j]\right| = n_j - \mu_j,$$

by Kingman's bound (Kingman 1962):

$$\sup_{B \geq 0} \mathbb{E}[\bar{W}^B_{(j)}] \leq 2n_j + \frac{\mu_j}{2(n_j - \mu_j)} < \infty.$$

Combining the zero-drift and negative-drift cases, under the nonpositive drift condition in (19),

$$\sum_{j=1}^{m} \mathbb{E}[\bar{W}_{(j)}^{B}] = O(B^{1/2}), \qquad \frac{1}{B}\sum_{b=0}^{B-1}\sum_{j=1}^{m} \mathbb{E}[\bar{W}_{(j)}^{b}] = O(B^{1/2}).$$

Under strict slack $\mu_j < n_j$ for all j, both quantities are $O(1)$.

It remains to translate these B-slot bounds into the theorem's finite-horizon metrics. The number of completed comparison slots in the physical horizon $[0,T]$ is exactly

$$B_\zeta(T) = \max\{b : u_b \le T\} = \left\lfloor \frac{\zeta T}{\Delta^\pi} \right\rfloor,$$

and the residual time is less than one comparison slot. Since $B_\zeta = \zeta T/\Delta^\pi + O(1)$, $B_\zeta^{-1/2} = O((\zeta T)^{-1/2})$ and $B_\zeta^{-1} = O((\zeta T)^{-1})$.

For throughput, terminal entry backlog controls the completion shortfall. More precisely, for the comparison policy,

$$\textbf{Throughput}^* - \mathbb{E}[\textbf{Throughput}^{(\zeta,\bar{\pi})}] \le \frac{1}{\zeta T}\sum_{j=1}^{m} l'_j\, \mathbb{E}[\bar{W}_{(j)}^{B_\zeta}] + O((\zeta T)^{-1}),$$

where the $O((\zeta T)^{-1})$ term accounts for the final partial slot and the bounded number of prompts already in the decode pipeline. This gives an $O((\zeta T)^{-1/2})$ gap under nonpositive drift and an $O((\zeta T)^{-1})$ gap under strict slack.

For delay, terminal backlog is not the right object; the relevant quantity is the time average of the entry queues. Let $\bar{Q}^b = \sum_{j=1}^{m} \bar{W}_{(j)}^{b}$. In physical time, each comparison slot has length Δ^π/ζ. The theorem, however, reports service-normalized delay, multiplying elapsed physical time by ζ. Thus each comparison slot contributes a constant scaled time Δ^π, and the waiting contribution is bounded by

$$O\left(\frac{1}{B_\zeta}\sum_{b=0}^{B_\zeta-1} \mathbb{E}[\bar{Q}^b]\right).$$

The prefix bound therefore yields

$$\mathbb{E}[\textbf{Latency}^{(\zeta,\bar{\pi})}], \mathbb{E}[\textbf{TTFT}^{(\zeta,\bar{\pi})}] = O(B_\zeta^{1/2}) = O((\zeta T)^{1/2})$$

under nonpositive drift. Under strict slack, the same expression is $O(1)$. The fixed number of service stages adds only a constant in service-normalized time. The comparison monotonicity transfers these throughput, latency, and TTFT bounds from $\bar{\pi}$ to the actual event-driven WAIT policy, completing the proof of Theorem 1.

Appendix E: Proof of Theorem 2

Proof Outline. This proof establishes asymptotic optimality of Nested WAIT when output lengths are unknown. The algorithm achieves *on-the-fly classification*: prompts classify themselves by their actual output lengths at segment boundaries, requiring no prediction. The structure proceeds as follows:

1. **First segment analysis**: All prompts enter segment 1 with Poisson arrivals. We couple with a Lindley process and apply Kingman's inequality for bounded expectations.

2. **Subsequent segments analysis**: After completing segment $k-1$, prompts naturally reveal whether they continue to segment k. This type revelation follows Binomial thinning: a fraction p_k of completions continue, while short prompts finish without being delayed by longer ones. We apply Lindley coupling to each segment.

3. **High-probability bounds prevent eviction**: Expected bounds ensure average stability, but stochastic fluctuations risk overflow. We construct an exponential martingale and apply Doob's inequality to obtain high-probability memory bounds, preventing eviction cascades.

We adopt notation consistent with the main text (Section 5). All prompt types have common prefill length l, and type-j prompts have output length l'_j. Let $\delta_1 = l'_1 + 1$, $\bar{s}_1 = l'_1/2$, and for $k \geq 2$, let $\delta_k = l'_k - l'_{k-1}$ and $\bar{s}_k = (l'_{k-1} + 1 + l'_k)/2$. We use b to denote the *batch index* (count of completed iterations) and $k \in \{1, \ldots, m\}$ to denote the *segment index*, where segment k contains prompts with output lengths in $(l'_{k-1}, l'_k]$. The threshold for segment k is n_k (from Equation (12)). We denote by $W^b_{(k)}$ the queue length for segment k after batch b, and by B the total number of batches over the time horizon. The thinning probability $p_k = (\sum_{j=k}^m \lambda_j)/(\sum_{j=k-1}^m \lambda_j)$ represents the fraction of prompts completing segment $k-1$ that continue to segment k.

E.1. First Segment Analysis

All prompts initially enter segment 1 with Poisson arrivals at aggregate rate $\sum_{j=1}^m \lambda_j$. We construct a dominating process and apply Kingman's inequality to bound expected queue length.

For the first segment ($k = 1$), the state transition is:

$$W^{b+1}_{(1)} = W^b_{(1)} + Y^b_{(1)} - n_1 \cdot \mathbf{1}\{W^b_{(1)} + Y^b_{(1)} \geq n_1\}, \quad Y^b_{(1)} \sim \text{Poisson}(\Delta T_b \cdot \sum_{j=1}^m \lambda_j),$$

where ΔT_b denotes the processing time of batch b. Since $\Delta T_b \le \Delta T_{[1,\dots,m]}$ (the time for a full batch), we construct a dominating process:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} \ge n_1\}, \quad \bar{Y}_{(1)}^{b} \sim \text{Poisson}(\Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^{m} \lambda_j).$$

This is further dominated by a Lindley process:

$$\tilde{W}_{(1)}^{0} = 2n_1, \quad \tilde{W}_{(1)}^{b+1} = \max\{2n_1, \tilde{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} - n_1\}.$$

Since $\Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^{m} \lambda_j < n_1$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E}\left[W_{(1)}^{b}\right] \le 2n_1 + \frac{\Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^{m} \lambda_j}{2\left(n_1 - \Delta T_{[1,\dots,m]} \cdot \sum_{j=1}^{m} \lambda_j\right)}.$$

Remark. Prompts that have not yet been admitted to prefill have no resident KV cache and therefore do not consume GPU KV-cache memory. This statement does not rely on swapping an already-prefilled cache out of GPU memory.

E.2. Subsequent Segments Analysis ($k \ge 2$)

For segment k ($2 \le k \le m$), the arrival process reflects on-the-fly classification: prompts that completed segment $k-1$ reveal whether their output lengths exceed l'_{k-1}. This type revelation follows binomial thinning with probability p_k.

Arrivals come from prompts that completed segment $k-1$:

$$Y_{(k)}^{b} \sim \text{Binomial}\left(n_{k-1},\, p_k\right), \quad \text{where } p_k = \frac{\sum_{j=k}^{m} \lambda_j}{\sum_{j=k-1}^{m} \lambda_j}.$$

On-the-fly Classification via Binomial Thinning. The arrival process $Y_{(k)}^{b} \sim \text{Binomial}(n_{k-1}, p_k)$ reflects the *on-the-fly classification mechanism*, not merely a modeling choice. After segment $k-1$ processing, exactly n_{k-1} prompts complete that stage. Among these, a fraction $p_k = \frac{\sum_{j=k}^{m} \lambda_j}{\sum_{j=k-1}^{m} \lambda_j}$ have output lengths exceeding l'_{k-1}. These prompts naturally continue to segment k. Prompts classify themselves by their actual output lengths—no prediction needed. Short prompts complete quickly without being delayed by longer ones.

The state transition is:

$$W_{(k)}^{b+1} = W_{(k)}^{b} + Y_{(k)}^{b} - n_k \cdot \mathbf{1}\{W_{(k)}^{b} + Y_{(k)}^{b} \ge n_k\},$$

where b indexes the batches containing segment $k-1$. Note that each segment has its own batch index, but all are bounded by the total batch count B.

Define $X^b_{(k)} = Y^b_{(k)} - n_k$. We construct a coupled dominating process:

LEMMA 7 (**Coupled Process for Segment** k). *Define* $\{\tilde{W}^b_{(k)}\}$ *by:*

$$\tilde{W}^0_{(k)} = n_k,$$
$$\tilde{W}^{b+1}_{(k)} = \max\{n_k, \tilde{W}^b_{(k)} + X^b_{(k)}\}.$$

Then $\tilde{W}^b_{(k)} \geq W^b_{(k)}$ *for all* $b \in \mathbb{N}$.

The proof is in Appendix G.4. The process $\{\tilde{W}^b_{(k)}\}$ has the Lindley representation:

$$\tilde{W}^{b+1}_{(k)} = n_k + \max_{0\leq i\leq b}\{S^i_{(k)}\},$$
$$\text{where } S^i_{(k)} = \sum_{r=1}^{i} X^r_{(k)},\ S^0_{(k)} = 0.$$

Expected Queue Length. Since $\mathbb{E}\left[X^b_{(k)}\right] = n_{k-1}p_k - n_k < 0$ (negative drift), by Kingman's inequality (Kingman 1962):

$$\mathbb{E}\left[W^b_{(k)}\right] \leq \mathbb{E}\left[\tilde{W}^b_{(k)}\right] \leq n_k + \frac{n_{k-1}\cdot p_k(1-p_k)}{2\left(n_k - n_{k-1}\cdot p_k\right)}.$$

Negative drift ensures queue stability, preventing unbounded growth that would lead to eviction. This bound ensures finite expected queue length for each segment $k \geq 2$, controlling expected GPU memory consumption. The segment structure ensures shorter prompts complete without waiting for longer ones to finish.

E.3. Throughput Gap Bound

Using B as the total batch count (upper bound for each segment's batch index), the throughput gap is:

$$\frac{1}{B}\sum_{k=1}^{m}\mathbb{E}\left[W^B_{(k)}\right] \leq \frac{2n_1}{B} + \frac{\Delta T_{[1,\ldots,m]}\cdot\sum_{j=1}^m\lambda_j}{2B\left(n_1 - \Delta T_{[1,\ldots,m]}\cdot\sum_{j=1}^m\lambda_j\right)} + \frac{1}{B}\sum_{k=2}^{m}n_k + \frac{1}{B}\sum_{k=2}^{m}\frac{n_{k-1}\cdot p_k(1-p_k)}{2\left(n_k - n_{k-1}\cdot p_k\right)}.$$

Asymptotic Regime. As $\zeta \to \infty$, the throughput gap is $O((\zeta B)^{-1})$. The expected latency and TTFT are $O(1)$.

E.4. High-Probability Memory Bound

High-Probability Bounds Prevent Eviction. Expected queue length bounds ensure average stability. However, stochastic fluctuations may cause $W^b_{(k)}$ to exceed these limits, risking memory overflow and eviction. We derive high-probability bounds to quantify this overflow risk. By choosing capacity C to ensure overflow probability $< \delta$ across all batches and segments, we prevent eviction cascades.

Martingale Construction. To obtain high-probability bounds on queue length, we need to control tail probabilities of the random walk maximum $\max_{0\le i\le s-1}\{S^i_{(k)}\}$. Expected bounds alone cannot prevent rare but catastrophic overflow events. We construct an exponential martingale that enables Doob's inequality, converting expected bounds into probabilistic guarantees.

We need to estimate

$$P\left(\max\{0, S^1_{(k)}, \cdots, S^{s-1}_{(k)}\} \ge c\right) = P\left(\max_{0\le i\le s-1}\{S^i_{(k)}\} \ge c\right), \forall c > 0, \text{ where } S^0_{(k)} = 0.$$

Define the exponential martingale $M^i_{(k)} = e^{\theta S^i_{(k)}}$. For this to be a martingale, the moment generating function must equal 1, ensuring zero expected drift in log space:

$$\phi_{(k)}(\theta) = \mathbb{E}\left[e^{\theta X^i_{(k)}}\right] = e^{-\theta n_k}(1 - p_k + p_k e^{\theta})^{n_{k-1}} = 1.$$

The following lemma establishes existence and properties of the solution $\theta_k > 0$ that satisfies this martingale condition (proof in Appendix G.5).

LEMMA 8. *Suppose $n_{k-1} > n_k > n_{k-1}p_k$ and $p_k \in (0,1)$. Then there exists a unique solution $\theta_k > 0$ to the equation*

$$e^{-\theta_k n_k}(1 - p_k + p_k e^{\theta_k})^{n_{k-1}} = 1, \tag{20}$$

where θ_k is strictly increasing with respect to $D = n_k - n_{k-1}p_k$ and satisfies the lower bound

$$\theta_k \ge \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}.$$

When $\theta_k > 0$ satisfies $\phi(\theta_k) = 1$, the process $M^i_{(k)}$ is a martingale starting at $M^0_{(k)} = e^{\theta_k \cdot 0} = 1$. This zero-drift property in log space enables Doob's inequality to bound tail probabilities.

By Doob's martingale inequality (Durrett 2019), we have that

$$P(\max_{1\le i\le b-1}\{e^{\theta_k S^i_{(k)}}\} \ge a) \le \frac{\mathbb{E}\left[M^i\right]}{a} = \frac{\mathbb{E}\left[M^0\right]}{a} = \frac{1}{a}.$$

Exponentiating both sides of the inequality event, we obtain:

$$P(\max_{1\le i\le b-1}\{e^{\theta S^i_{(k)}}\}\ge e^{\theta c})\le\frac{1}{e^{\theta c}}\Rightarrow P(\max_{1\le i\le b-1}\{S^i_{(k)}\}\ge c)\le e^{-\theta c}.$$

Using the dominance $\tilde{W}^b_{(k)}=n_k+\max_{0\le i\le b-1}\{S^i_{(k)}\}\ge W^b_{(k)}$, we have

$$P\left(W^b_{(k)}\ge c\right)<P\left(\tilde{W}^b_{(k)}\ge c\right)=P(\max_{1\le i\le b-1}\{S^i_{(k)}\}\ge c-n_k)\le e^{-\theta_k(c-n_k)}$$

For all $2\le k\le m$ and $1\le b\le B$:

$$P\left(W^b_{(k)}\ge c\right)<e^{-\theta_k(c-n_k)},$$

where $\theta_k>0$ is the solution to (20).

Union bound for safety guarantee. To ensure memory consumption does not exceed the bound at any batch $b\in\{1,\dots,B\}$ with probability at least $1-\delta$, we apply a union bound across all segments and batches. For each segment $k\in\{2,\dots,m\}$ and batch index $b\in\{1,\dots,B\}$, we have:

$$P\left(W^b_{(k)}\ge c\right)<e^{-\theta_k(c-n_k)},$$

There are $m-1$ segments (from $k=2$ to m) and B batches (from $b=1$ to B). We allocate the total failure probability δ across all segments and batches. The total number of events is $(m-1)\cdot B$. Set the overflow probability for each segment and batch to:

$$P\left(W^b_{(k)}\ge c_k\right)\le\frac{\delta}{(m-1)\cdot B}\Rightarrow e^{-\theta_k(c_k-n_k)}\le\frac{\delta}{(m-1)\cdot B}\Rightarrow c_k\ge n_k+\frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k}$$

If we choose

$$c_k=n_k+\frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k},$$

then we have that

$$P\left(W^b_{(k)}\ge c_k\right)\le\frac{\delta}{(m-1)\cdot B}.$$

Next, we compute the joint probability of overflow across all segments and batches using a union bound:

$$P\left(\bigcup_{k=2}^{m}\bigcup_{b=1}^{B}\{W^b_{(k)}\ge c_k\}\right)\le\sum_{k=2}^{m}\sum_{b=1}^{B}P\left(W^b_{(k)}\ge c_k\right)\le(m-1)\cdot B\cdot\frac{\delta}{(m-1)\cdot B}=\delta$$

Thus:

$$P\left(W_{(k)}^b \le c_k \text{ for all } \begin{matrix} k \in \{2,\dots,m\}, \\ b \in \{1,\dots,B\} \end{matrix}\right) \ge 1-\delta.$$

So the memory consumption is bounded:

$$\text{Memory Consumption}^b = M^\pi + \sum_{k=2}^{m} W_{(k)}^b \cdot (l + l'_{k-1}) \le M^\pi + \sum_{k=2}^{m} c_k \cdot (l + l'_{k-1})$$

where M^π is the peak batch memory usage as defined in Equation (13):

$$M^\pi = \sum_{j=1}^{m} n_j (l + \bar{s}_j)\delta_j.$$

Note that this differs from the M^π formula for the WAIT algorithm (Theorem 1), because Nested WAIT groups prompts into segments based on their unknown output lengths, leading to segment-wise memory accumulation.

Substituting $c_k = n_k + \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k}$, we have that

$$\text{Memory Consumption}^b \le M^\pi + \sum_{k=2}^{m} \left(n_k + \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k} \right) \cdot (l + l'_{k-1}).$$

Define the memory bound $\mathbf{M}^B$:

$$\mathbf{M}^B = M^\pi + \sum_{k=2}^{m} n_k \cdot (l + l'_{k-1}) + \sum_{k=2}^{m} \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k} \cdot (l + l'_{k-1}).$$

By the lower bound in Lemma 8, we can obtain the upper bound of the third term:

$$\sum_{k=2}^{m} \frac{\ln\left(\frac{(m-1)\cdot B}{\delta}\right)}{\theta_k} \cdot (l + l'_{k-1}) \le \sum_{k=2}^{m} \frac{n_{k-1}}{8(n_k - n_{k-1}p_k)} \ln\left(\frac{(m-1)\cdot B}{\delta}\right) \cdot (l + l'_{k-1})$$

And this $\mathbf{M}^B$ ensures that with probability at least $1-\delta$, the memory consumption does not exceed $\mathbf{M}^B$ at any batch index $b \in \{1,\dots,B\}$.

Appendix F: Proof of Theorem 3

Proof Outline. This proof extends Nested WAIT to time-varying arrival rates $\lambda_j(t)$. Time-varying rates make exact load balance impossible, since the optimal threshold depends on the instantaneous arrival pattern. The *worst-case domination strategy* maintains load balance even at peak rates: by designing thresholds to handle worst-case arrivals, we ensure system stability across all scenarios. This conservative approach prevents eviction throughout the time horizon. The structure proceeds as follows:

1. **First segment analysis**: Poisson arrivals with time-varying rates are dominated by a time-invariant process using worst-case aggregate rate $\mathbf{\Lambda}^{\pi} = \sup_b\{\sum_{j=1}^{m} \lambda_j[t(b), t(b) + \Delta T_{[1,\ldots,m]}]\}$. We couple with a Lindley process and apply Kingman's inequality.

2. **Subsequent segments analysis**: Binomial arrivals with time-varying thinning probabilities $p_k(b)$ are dominated by worst-case $p_k^* = \sup_b\{p_k(b)\}$. This ensures safety across all batch compositions. We apply Lindley coupling to each segment.

3. **Throughput and high-probability bounds**: Asymptotic optimality follows from the time-invariant dominated processes. High-probability memory bounds follow the same martingale construction, using worst-case parameters to prevent eviction.

We use b to denote the *batch index* and $k \in \{1, \ldots, m\}$ to denote the *segment index*. The total number of batches is denoted by B.

F.1. First Segment Analysis

For the first segment ($k = 1$), the state transition rule is

$$W_{(1)}^{b+1} = W_{(1)}^{b} + Y_{(1)}^{b} - n_1 \cdot \mathbf{1}\{W_{(1)}^{b} + Y_{(1)}^{b} \geq n_1\}, \quad Y_{(1)}^{b} \sim \text{Poisson}\Big(\sum_{j=1}^{m} \lambda_j[t(b), t(b) + \Delta T_b]\Big),$$

where $t(b)$ denotes the start time of batch b and ΔT_b denotes the processing time of batch b.

Worst-case domination. Time-varying arrival rates prevent exact load balance since optimal thresholds depend on instantaneous arrival patterns. To maintain stability across all scenarios, we use worst-case domination: design thresholds to handle peak arrival rates, ensuring eviction prevention even under worst-case conditions. Since $\Delta T_b \leq \Delta T_{[1,\ldots,m]}$ (the time for a full batch containing all m types), we construct a dominating process $\{\bar{W}_{(1)}^{b}\}$:

$$\bar{W}_{(1)}^{b+1} = \bar{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} - n_1 \cdot \mathbf{1}\{\bar{W}_{(1)}^{b} + \bar{Y}_{(1)}^{b} \geq n_1\}, \quad \bar{Y}_{(1)}^{b} \sim \text{Poisson}\Big(\sum_{j=1}^{m} \lambda_j[t(b), t(b) + \Delta T_{[1,\ldots,m]}]\Big).$$

By (15), the aggregate arrival rate during any batch is bounded by the worst-case aggregate rate:

$$\sum_{j=1}^{m} \lambda_j[t(b), t(b) + \Delta T_{[1,\ldots,m]}] \leq \sup_{\forall b}\left\{\sum_{j=1}^{m} \lambda_j[t(b), t(b) + \Delta T_{[1,\ldots,m]}]\right\} < n_1.$$

Define $\mathbf{\Lambda}^{\pi} = \sup_{\forall b}\left\{\sum_{j=1}^{m} \lambda_j[t(b), t(b) + \Delta T_{[1,\ldots,m]}]\right\} < n_1$ as the worst-case aggregate arrival rate. This enables a time-invariant dominating process that provides tractable bounds:

$$\tilde{W}_{(1)}^{b+1} = \tilde{W}_{(1)}^{b} + \tilde{Y}_{(1)}^{b} - n_1 \cdot \mathbf{1}\{\tilde{W}_{(1)}^{b} + \tilde{Y}_{(1)}^{b} \geq n_1\}, \quad \tilde{Y}_{(1)}^{b} \sim \text{Poisson}(\mathbf{\Lambda}^{\pi}).$$

This time-invariant process enables classical queueing analysis. Define $X^b_{(1)} = \tilde{Y}^b_{(1)} - n_1$. The process $\{\tilde{W}^b_{(1)}\}$ is dominated by a Lindley process:

$$\overline{W}^0_{(1)} = 2n_1, \quad \overline{W}^{b+1}_{(1)} = \max\{2n_1, \overline{W}^b_{(1)} + X^b_{(1)}\}, \quad \forall b \in \mathbb{N}.$$

Thus we have the dominance chain:

$$W^b_{(1)} \le \bar{W}^b_{(1)} \le \tilde{W}^b_{(1)} \le \overline{W}^b_{(1)}, \quad \forall b \in \mathbb{N}.$$

Since $n_1 > \mathbf{\Lambda}^\pi$, the process $\{\overline{W}^b_{(1)}\}$ has negative drift. By Kingman's inequality (Kingman 1962), with

$$\mathrm{Var}(X^b_{(1)}) = \mathbf{\Lambda}^\pi, \quad \left|\mathbb{E}\left[X^b_{(1)}\right]\right| = n_1 - \mathbf{\Lambda}^\pi,$$

we obtain

$$\mathbb{E}\left[W^b_{(1)}\right] \le 2n_1 + \frac{\mathbf{\Lambda}^\pi}{2\,(n_1 - \mathbf{\Lambda}^\pi)}.$$

F.2. Subsequent Segments Analysis ($k \ge 2$)

For segment k ($2 \le k \le m$), arrivals come from prompts that completed segment $k-1$. The batch index b for segment k counts only batches containing segment $k-1$. These indices differ across segments but are bounded by the total batch count B.

Time-varying thinning probabilities. Consider the b-th arrival to segment k, which comes from the b-th batch containing segment $k-1$. The thinning probability $p_k(b)$ depends on the prompt composition in this batch and varies over time. Although exact arrival times are unknown, arrivals occur within some interval $[t, t+\Delta t]$ where $t + \Delta t \le t(b, k-1)$, the starting time of the b-th batch containing segment $k-1$. To handle this time variation, we use the worst-case thinning probability. By (15):

$$p_k(b) = \frac{\sum_{j=k}^m \lambda_j[t, t+\Delta t]}{\sum_{j=k-1}^m \lambda_j[t, t+\Delta t]} \le p_k^* < \frac{n_k}{n_{k-1}}, \quad \forall b \in \mathbb{N}.$$

The arrival process is

$$Y^b_{(k)} \sim \mathrm{Binomial}(n_{k-1}, p_k(b)), \quad \forall b \ge 0,$$

and the state transition rule is

$$W^{b+1}_{(k)} = W^b_{(k)} + Y^b_{(k)} - n_k \cdot \mathbf{1}\{W^b_{(k)} + Y^b_{(k)} \ge n_k\},$$

where b indexes batches containing segment $k-1$, since new arrivals to segment k occur only when segment $k-1$ is processed.

Define $X^b_{(k)} = Y^b_{(k)} - n_k$. Although the distribution $Y^b_{(k)} \sim \mathrm{Binomial}(n_{k-1}, p_k(b))$ is time-varying, the coupling construction follows Lemma 7:

LEMMA 9 **(Coupling for Time-Varying Case).** *Define a coupled process* $\{\tilde{W}^b_{(k)}\}$ *by:*

$$\begin{aligned}\tilde{W}^0_{(k)} &= n_k,\\ \tilde{W}^{b+1}_{(k)} &= \max\{n_k,\, \tilde{W}^b_{(k)} + X^b_{(k)}\},\\ X^b_{(k)} &= Y^b_{(k)} - n_k.\end{aligned}$$

Then $\tilde{W}^b_{(k)} \geq W^b_{(k)}$ *for all* $b \in \mathbb{N}$.

To apply Kingman's inequality (Kingman 1962), we construct a time-invariant dominating process:

$$\begin{aligned}\bar{W}^0_{(k)} &= n_k,\\ \bar{W}^{b+1}_{(k)} &= \max\{n_k,\, \bar{W}^b_{(k)} + \bar{X}^b_{(k)}\},\\ \bar{X}^b_{(k)} &= \bar{Y}^b_{(k)} - n_k,\\ \bar{Y}^b_{(k)} &\sim \text{Binomial}(n_{k-1}, p^*_k), \quad \forall b \in \mathbb{N}.\end{aligned}$$

Then we have the dominance chain:

$$\bar{W}^b_{(k)} \geq \tilde{W}^b_{(k)} \geq W^b_{(k)}, \quad \forall b \in \mathbb{N}.$$

The process $\bar{W}^b_{(k)}$ has the Lindley representation:

$$\begin{aligned}\bar{W}^0_{(k)} &= n_k,\\ \bar{W}^{b+1}_{(k)} &= n_k + \max_{0\leq i\leq b}\{\bar{S}^i_{(k)}\},\\ \text{where } \bar{S}^0_{(k)} = 0,\ \bar{S}^i_{(k)} &= \sum_{r=1}^{i} \bar{X}^r_{(k)},\ 1 \leq i \leq b.\end{aligned}$$

Since

$$\mathbb{E}\left[\bar{X}^b_{(k)}\right] = n_{k-1}\cdot p^*_k - n_k < 0, \quad \forall b \in \mathbb{N},$$

the coupled process $\{\bar{W}^b_{(k)}\}$ has negative drift. With

$$\text{Var}(\bar{X}^b_{(k)}) = n_{k-1}p^*_k(1-p^*_k), \quad \left|\mathbb{E}\left[\bar{X}^b_{(k)}\right]\right| = n_k - p^*_k n_{k-1},$$

Kingman's inequality (Kingman 1962) yields:

$$\mathbb{E}\left[\max_{0\leq i\leq b}\{\bar{S}^i_{(k)}\}\right] \leq \frac{n_{k-1}p^*_k(1-p^*_k)}{2(n_k - n_{k-1}p^*_k)}.$$

Thus, the expected queue length is bounded:

$$\mathbb{E}\left[W^b_{(k)}\right] \leq \mathbb{E}\left[\tilde{W}^b_{(k)}\right] \leq \mathbb{E}\left[\bar{W}^b_{(k)}\right] \leq n_k + \frac{n_{k-1}p^*_k(1-p^*_k)}{2(n_k - n_{k-1}p^*_k)}.$$

F.3. Throughput and High-Probability Bounds

The discussion of asymptotic optimal throughput under bounded latency and TTFT follows Appendix E. For high-probability bounds, the analysis follows the same martingale construction applied to the time-invariant coupled process $\{\bar{W}^b_{(k)}\}$ defined above. By the same argument as in Appendix E, with $\bar{X}^r_{(k)} = \bar{Y}^r_{(k)} - n_k$, $\bar{Y}^r_{(k)} \sim \text{Binomial}(n_{k-1}, p^*_k)$, and $p^*_k < n_k/n_{k-1}$, the memory bound $\mathbf{M}^B$ is:

$$\mathbf{M}^B = M^\pi + \sum_{k=2}^{m} n_k \cdot (l + l'_{k-1}) + \sum_{k=2}^{m} \theta_k^{-1} \ln\left(\frac{(m-1)\cdot B}{\delta}\right) \cdot (l + l'_{k-1}),$$

where θ_k $(2 \le k \le m)$ is the unique positive solution to:

$$e^{-\theta_k n_k}(1 - p^*_k + p^*_k e^{\theta_k})^{n_{k-1}} = 1. \tag{21}$$

Since $p^*_k = \sup_b\{p_k(b)\}$ represents the worst-case thinning probability, by the strictly increasing property of θ_k in Lemma 8, this yields the smallest θ_k and hence the largest memory bound term. The lower bound for θ_k follows from the same analysis in Appendix E.

Appendix G: Proof of Lemmas

Throughout this section, we use b for the batch index and B for the total number of batches.

G.1. Proof of Lemma 1

To prove this lemma, we take expectations on both sides of the state transition equation:

$$\mathbb{E}\left[W^{b+1}\right] = \mathbb{E}\left[W^b\right] + \lambda - \lambda\mathbb{P}(W^b + X^b \ge \lambda).$$

Summing from $b = 0$ to $B - 1$, and noting that $W^0 = 0$, we obtain:

$$\mathbb{E}\left[W^B\right] = \lambda B - \lambda\mathbb{E}\left[B - B_{\text{stuck}}\right] = \lambda \cdot \mathbb{E}\left[B_{\text{stuck}}\right].$$

G.2. Proof of Lemma 2

We prove by induction on the batch index b. The coupled process $\{\tilde{W}^b\}$ starts with a safety buffer of 2λ (compared to the real process starting at 0), ensuring it remains above $W^b + \lambda$ throughout. For the inductive step, we consider two cases based on whether the system can process a batch or experiences a stuck iteration.

Base case. For $b = 0$: $\tilde{W}^0 = 2\lambda \ge \lambda = W^0 + \lambda$.

Inductive step. Assume $\tilde{W}^b \geq W^b + \lambda$ for some $b \geq 0$. We show $\tilde{W}^{b+1} \geq W^{b+1} + \lambda$ by considering two cases:

Case 1 (Batch processed): If $W^b + X^b \geq \lambda$, the queue has accumulated enough prompts to process a batch, so $W^{b+1} = W^b + X^b - \lambda$. Then

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq \tilde{W}^b + X^b - \lambda \geq (W^b + \lambda) + X^b - \lambda = W^{b+1} + \lambda.$$

Case 2 (Stuck iteration): If $W^b + X^b < \lambda$, insufficient prompts are available and the system cannot process a batch (stuck iteration). The queue simply accumulates arrivals: $W^{b+1} = W^b + X^b$. The safety buffer ensures dominance:

$$\tilde{W}^{b+1} = \max\{2\lambda, \tilde{W}^b + X^b - \lambda\} \geq 2\lambda = \lambda + \lambda > W^{b+1} + \lambda.$$

By induction, $\tilde{W}^b \geq W^b + \lambda$ for all $b \in \mathbb{N}$.

G.3. Proof of Lemma 6

The lemma compares the type-j entry queue in the periodic comparison system with a reflected random walk that always subtracts n_j after adding one slot of arrivals, but never falls below the safety level $2n_j$. Fix type j and write $\bar{W}^b = \bar{W}^b_{(j)}$, $\widetilde{W}^b = \widetilde{W}^b_{(j)}$, $n = n_j$, and $\bar{X}^b = \bar{X}^b_{(j)}$. We prove by induction that $\widetilde{W}^b \geq \bar{W}^b$.

The claim holds at $b = 0$, since $\widetilde{W}^0 = 2n$ and $\bar{W}^0 = 0$. Suppose $\widetilde{W}^b \geq \bar{W}^b$. If $\bar{W}^b + \bar{X}^b \geq n$, then the comparison queue serves n prompts and

$$\bar{W}^{b+1} = \bar{W}^b + \bar{X}^b - n.$$

Using the induction hypothesis,

$$\widetilde{W}^{b+1} = \max\{2n, \widetilde{W}^b + \bar{X}^b - n\} \geq \widetilde{W}^b + \bar{X}^b - n \geq \bar{W}^{b+1}.$$

If $\bar{W}^b + \bar{X}^b < n$, then no type-$j$ service occurs in the comparison queue and $\bar{W}^{b+1} = \bar{W}^b + \bar{X}^b < n$. In this case $\widetilde{W}^{b+1} \geq 2n > \bar{W}^{b+1}$. Thus the domination holds for all b, and the argument applies independently to every type j.

G.4. Proof of Lemma 7

The proof follows the same induction structure as Lemma 2, adapted to the Nested WAIT setting where arrivals follow binomial thinning from segment $k-1$ completions.

Base case. For $b = 0$: $\tilde{W}^0_{(k)} = n_k \geq 0 = W^0_{(k)}$.

Inductive step. Assume $\tilde{W}^b_{(k)} \geq W^b_{(k)}$ for some $b \geq 0$. We show $\tilde{W}^{b+1}_{(k)} \geq W^{b+1}_{(k)}$ by case analysis:

Case 1 (Segment batch processed): If $W^b_{(k)} + Y^b_{(k)} \geq n_k$, then $W^{b+1}_{(k)} = W^b_{(k)} + Y^b_{(k)} - n_k$, so

$$\tilde{W}^{b+1}_{(k)} = \max\{n_k, \tilde{W}^b_{(k)} + X^b_{(k)}\} \geq \tilde{W}^b_{(k)} + X^b_{(k)} \geq W^b_{(k)} + Y^b_{(k)} - n_k = W^{b+1}_{(k)}.$$

Case 2 (Segment stuck): If $W^b_{(k)} + Y^b_{(k)} < n_k$, then $W^{b+1}_{(k)} = W^b_{(k)} + Y^b_{(k)}$, so

$$\tilde{W}^{b+1}_{(k)} = \max\{n_k, \tilde{W}^b_{(k)} + X^b_{(k)}\} \geq n_k > W^{b+1}_{(k)}.$$

By induction, $\tilde{W}^b_{(k)} \geq W^b_{(k)}$ for all $b \in \mathbb{N}$.

G.5. Proof of Lemma 8

This lemma establishes existence and properties of the parameter $\theta_k > 0$ that makes the exponential process $e^{\theta_k S^i_{(k)}}$ a martingale. This enables Doob's inequality to convert expected queue length bounds into high-probability bounds, preventing eviction cascades.

Existence and uniqueness. For simplicity, we denote $\theta = \theta_k$. Define the function

$$f(\theta) = e^{-\theta n_k}(1 - p_k + p_k e^{\theta})^{n_{k-1}}.$$

We seek $\theta > 0$ such that $f(\theta) = 1$. First, evaluate $f(\theta)$ at $\theta = 0$:

$$f(0) = e^0(1 - p_k + p_k \cdot 1)^{n_{k-1}} = 1.$$

Thus, $\theta = 0$ is a trivial solution, but we need $\theta > 0$ to obtain useful high-probability bounds. To analyze $f(\theta)$ for $\theta > 0$, we consider the logarithm

$$g(\theta) = \ln f(\theta) = -\theta n_k + n_{k-1} \ln(1 - p_k + p_k e^{\theta}),$$

$$g'(\theta) = -n_k + n_{k-1} \cdot \frac{p_k e^{\theta}}{1 - p_k + p_k e^{\theta}}.$$

At $\theta = 0$,

$$g(0) = 0,\ g'(0) = -n_k + n_{k-1} p_k.$$

Since $\mathbb{E}[X^b_{(k)}] = n_{k-1} p_k - n_k < 0$, we have $g'(0) < 0$, so $f(\theta)$ is decreasing near $\theta = 0$. Consider the second derivative:

$$g''(\theta) = n_{k-1} \cdot \frac{p_k e^{\theta}(1 - p_k)}{(1 - p_k + p_k e^{\theta})^2} > 0 \quad \text{for} \quad \theta > 0,$$

since $p_k \in (0,1)$ and $n_{k-1} > 0$. Thus, $g(\theta)$ is strictly convex. Convexity ensures that $g(\theta)$ can cross zero at most once for $\theta > 0$, guaranteeing uniqueness. Now, consider the limit as $\theta \to \infty$:

$$f(\theta) \approx p_k^{n_{k-1}} e^{\theta(n_{k-1}-n_k)}.$$

By our settings, $n_{k-1} > n_k$, which ensures $f(\theta) \to \infty$ as $\theta \to \infty$. Thus, $f(\theta)$ is continuous, convex, starts at $f(0) = 1$, decreases below 1 for small $\theta > 0$ (since $g'(0) = n_{k-1}p_k - n_k < 0$ by negative drift), and increases to infinity. By the intermediate value theorem and convexity, there exists a unique $\theta > 0$ such that $f(\theta) = 1$.

Monotonicity in drift. Next, we prove that the solution $\theta > 0$ is strictly increasing with respect to the drift $D = n_k - n_{k-1}p_k$. This shows that tighter capacity (larger D, more negative drift) leads to larger θ and thus tighter high-probability bounds. Recall that $g(0) = 0$, $g'(0) = -D < 0$, and $g''(\theta) > 0$ for $\theta > 0$, so $g(\theta)$ is strictly convex. The unique positive solution $\theta > 0$ occurs where $g(\theta)$ crosses zero after initially decreasing from $g(0) = 0$.

Consider two values D_1 and D_2 such that $D_1 < D_2$, with corresponding functions $g_1(\theta) = -\theta(n_{k-1}p_k + D_1) + n_{k-1}\ln(1 - p_k + p_k e^\theta)$ and $g_2(\theta) = -\theta(n_{k-1}p_k + D_2) + n_{k-1}\ln(1 - p_k + p_k e^\theta)$, and solutions θ_1 and θ_2, respectively. At $\theta = 0$, we have $g_1'(0) = -D_1 > g_2'(0) = -D_2$, meaning $g_2(\theta)$ decreases more steeply from zero than $g_1(\theta)$.

For a fixed $\theta > 0$, the partial derivative $\frac{\partial g}{\partial D} = -\theta < 0$, so increasing D decreases $g(\theta)$ pointwise. Since $g(\theta)$ is convex and approaches infinity as $\theta \to \infty$, the zero crossing of $g(\theta; D)$ shifts to a larger θ when D increases. Hence, if $D_2 > D_1$, then $\theta_2 > \theta_1$. Operationally, this means larger safety margins yield tighter high-probability bounds.

Drift-to-θ scaling. Finally, we characterize how the solution $\theta > 0$ scales with drift D, n_{k-1}, and p_k. When the drift $D = n_k - n_{k-1}p_k$ is small (system near critical load), the solution θ is also small. To derive an explicit approximation, we expand $g(\theta)$ around $\theta = 0$ using Taylor's theorem:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(0)\theta^2 + O(\theta^3) = 0 - D\theta + \frac{1}{2}n_{k-1}p_k(1-p_k)\theta^2 + O(\theta^3).$$

Setting $g(\theta) = 0$ and neglecting higher-order terms, we obtain the quadratic approximation

$$-D\theta + \frac{1}{2}n_{k-1}p_k(1-p_k)\theta^2 \approx 0 \implies \theta \approx \frac{2D}{n_{k-1}p_k(1-p_k)}.$$

This shows that for small D, the solution scales linearly: $\theta = O(D)$. Specifically, $\theta \approx \frac{2(n_k - n_{k-1}p_k)}{n_{k-1}p_k(1-p_k)}$. This linear scaling implies that the high-probability memory bound grows logarithmically in B/δ, with the coefficient inversely proportional to the drift D.

General lower bound. To obtain a uniform lower bound valid for all $D > 0$ (not just small D), we apply Taylor's theorem with remainder:

$$g(\theta) = g(0) + g'(0)\theta + \frac{1}{2}g''(c)\theta^2 = -D\theta + \frac{1}{2}g''(c)\theta^2 = 0 \implies \theta = \frac{2D}{g''(c)}. \quad \text{for some } c \in (0, \theta)$$

To find an upper bound for $g''(c)$, we write $g''(x) = n_{k-1} \cdot \frac{p_k e^x (1-p_k)}{(1-p_k+p_k e^x)^2}$ and define $h(y) = \frac{p_k y(1-p_k)}{(1-p_k+p_k y)^2}$ where $y = e^x \geq 1$. By differentiation, $h(y)$ achieves a maximum of $\frac{1}{4}$ over $y \geq 1$ and $0 < p_k < 1$ (attained when the numerator and denominator are balanced). Thus we have that $g''(x) \leq n_{k-1} \cdot \frac{1}{4} = \frac{n_{k-1}}{4} \implies \theta = \frac{2D}{g''(c)} \geq \frac{2D}{n_{k-1}/4} = \frac{8D}{n_{k-1}}$. So a general lower bound is:

$$\theta \geq \frac{8(n_k - n_{k-1}p_k)}{n_{k-1}}. \tag{22}$$

Appendix H: Additional Experiments

This appendix reports supplemental robustness and implementation-fidelity analyses for Section 6. Section H.1 isolates the role of the accumulation requirement by comparing WAIT with a threshold-only variant. Section H.2 validates the simulator against direct GPU measurements and reports physical-GPU counterparts of the main single-type and real-data comparisons. Section H.3 examines repeated-run variability near the stability boundary, and Section H.4 evaluates the policy in a prefill-decode-disaggregated deployment.

H.1. Threshold without waiting

We isolate the role of waiting on the single-type `p512d20` workload by comparing full WAIT with a threshold-only variant, denoted *WAIT-no-wait*. For each arrival rate, the two policies use the same parameter setting and hence the same threshold upper bound. The threshold-only variant enforces this upper bound but does not wait for enough requests to accumulate before service. The comparison therefore holds the admission-capacity scale fixed and changes only whether service is delayed to form a full threshold batch.

Figure 17 reports the comparison across arrival rates. At low arrival rates, waiting introduces a modest latency cost: requests may wait to accumulate even when the server has enough capacity to process them immediately. This cost shrinks as the arrival rate increases. Near the overload boundary, waiting becomes useful because it regulates the timing at which work enters service, reducing latency relative to the threshold-only variant. Once the system is overloaded, the upper bound becomes the dominant control, and the two variants again behave similarly.

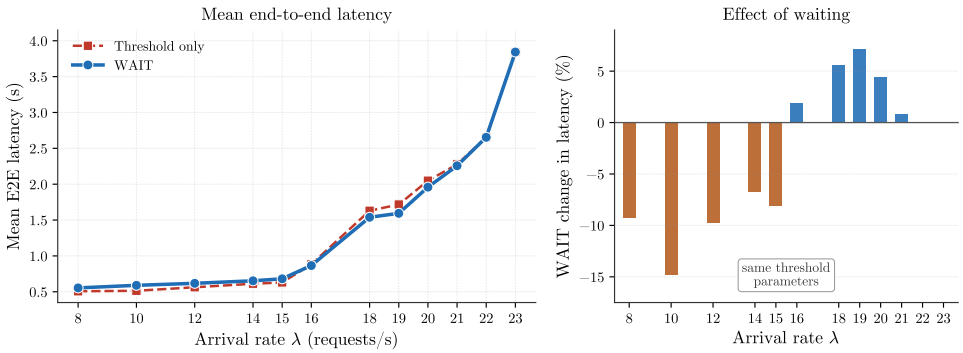

Figure 17 Effect of waiting on the single-type `p512d20` Vidur workload. At each arrival rate, full WAIT and the threshold-only variant use the same parameter setting; the comparison only changes whether service waits for enough requests to accumulate before scheduling.

H.2. Real-GPU validation

We complement the Vidur experiments in Section 6 with measurements on a physical NVIDIA A100 80 GB serving Llama-2-7B. These runs serve two purposes. First, direct GPU measurements calibrate the iteration-time model used by Vidur. Second, we implement WAIT's admission logic using SGLang 0.5.7, an open-source LLM serving framework (SGLang Team 2024), and test the resulting scheduler in end-to-end GPU runs. We report three comparisons: iteration-time measurements against Vidur's prediction, a single-type GPU workload corresponding to Section 6.1, and a real-data GPU workload corresponding to Section 6.2.

Simulator vs GPU. To calibrate the simulator in the operating region used in Section 6, we compare Vidur's iteration-time predictions with direct measurements on the same A100 80 GB GPU running Llama-2-7B under the SGLang baseline scheduler. The comparison uses the powers-of-two batch-size grid $B \in \{1, 2, 4, \ldots, 256\}$, with prefill length 256 and decode length 20. In Figure 18, each point is one batch-size configuration: the horizontal coordinate is the measured GPU iteration time, and the vertical coordinate is Vidur's prediction. Points near the $y = x$ line therefore indicate accurate simulator predictions. On this grid, the fitted linear iteration-time model has mean absolute percentage error 1.93% and $R^2 = 0.9943$. The fitted intercept and slope are $d_0 \approx 276$ ms and $d_1 \approx 0.0115$ ms per KV-cache token, respectively, consistent with the time model in Equation (1). This agreement supports the use of Vidur for the latency and stability comparisons in Section 6.

Single-type workload on GPU. Using the same single-type workload as Section 6.1, we run end-to-end SGLang experiments with Poisson arrivals at $\lambda \in \{1, 2, \ldots, 15\}$ requests/s and 500 prompts per rate. The SGLang baseline scheduler uses `chunked_prefill_size`= 256 and

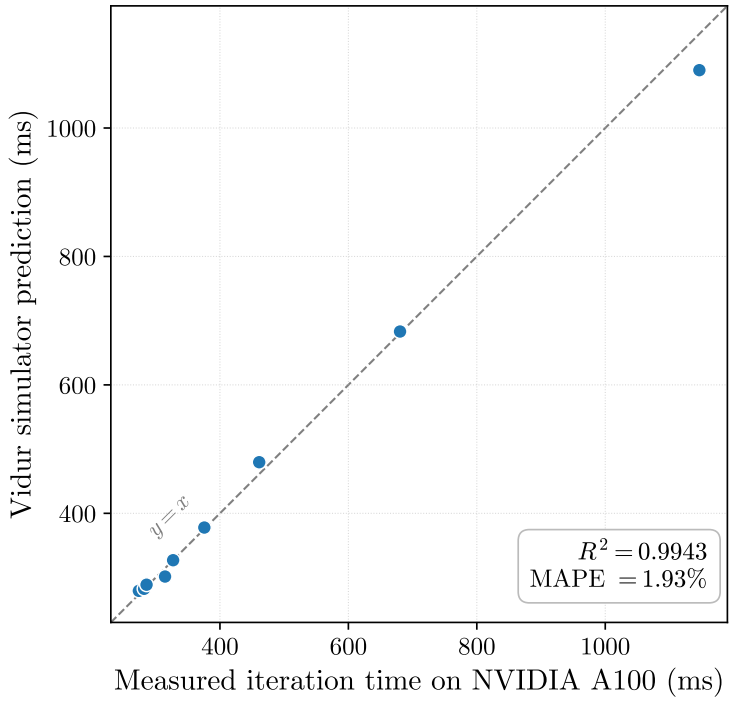

Figure 18 Simulator calibration against NVIDIA A100 80 GB GPU measurements. Each marker is one batch-size configuration: its horizontal coordinate is the measured GPU iteration time, and its vertical coordinate is the Vidur prediction. The comparison uses prefill length 256 and decode length 20 while varying B over the grid $\{1, 2, 4, \ldots, 256\}$. The dashed line is the $y = x$ reference. The fitted linear iteration-time model in Equation (1) achieves $R^2 = 0.9943$ and mean absolute percentage error 1.93% on the plotted grid.

`prefill_max_requests`= 8; its default configuration triggers out-of-memory failures at several mid-to-high rates and is therefore not a feasible baseline on this workload. WAIT uses the same parameter values at all rates. Its system-wide batch-size cap is tl = 21, and its per-stage WAIT threshold is $n_1 = 1$. The chunk size is 384, so a prefill of length 512 is processed as $K = \lceil 512/384 \rceil = 2$ prefill blocks before the request enters the 20 decode-token stages. Thus, the single-type pipeline has $K + 20 = 22$ stages: WAIT schedules at most one request from each active stage, while tl = 21 caps how many requests can be served concurrently across these stages. The SGLang patch implements this policy through the batch-size cap and the per-request prefill chunk cap; a transition block of size 10 switches the scheduler back to SGLang's native rule in lightly loaded states. Figure 19 reports mean end-to-end latency as λ varies. Under this GPU configuration, WAIT is consistently lower-latency than the feasible SGLang baseline scheduler, with an average reduction of 29.7% across the arrival-rate grid.

Real dataset on GPU. Using the same lmsys-chat-1m prompt distribution as Section 6.2, we run paired SGLang experiments with Poisson arrivals at $\lambda \in \{0.1, 0.2, \ldots, 0.7\}$ requests/s and 200

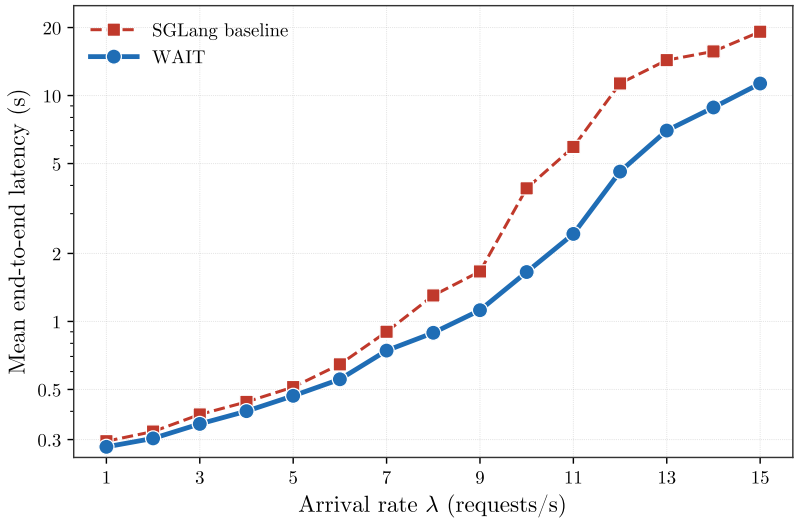

Figure 19 Single-type GPU experiment on SGLang 0.5.7 (NVIDIA A100 80 GB, Llama-2-7B, random prompts with input length 512 and output length 20): mean end-to-end latency versus arrival rate. WAIT is compared with the feasible SGLang baseline scheduler configuration used in this experiment and yields a mean latency reduction of 29.7% across the arrival-rate grid.

prompts per rate. The server is restarted before each rate so that the SGLang baseline and WAIT are compared from the same clean GPU state. WAIT uses chunk size 384 and a rate-specific system-wide batch-size cap tl. For the real-data runs, tl is distributed over the decode horizon through the listed segment partitions, using the same continuation-based allocation described in Section 6.2. Table 3 reports the batch-size caps and decode-stage partitions used in these GPU runs. Figure 20 reports mean end-to-end latency in absolute units. Across the tested rates, WAIT has lower latency than the SGLang baseline, with the largest reduction at $\lambda = 0.6$ and an average reduction of 3.6%.

λ (req/s)	tl	Decode-stage segments
0.1	40	$[0, 175), [175, 350), [350, 500]$
0.2	50	$[0, 210), [210, 420), [420, 500]$
0.3	60	$[0, 200), [200, 400), [400, 500]$
0.4	70	$[0, 175), [175, 350), [350, 500]$
0.5	70	$[0, 200), [200, 400), [400, 500]$
0.6	70	$[0, 175), [175, 350), [350, 500]$
0.7	95	$[0, 250), [250, 500]$

Table 3 WAIT parameter settings for the real-data GPU experiment on SGLang 0.5.7 with the lmsys-chat-1m prompt distribution, NVIDIA A100 80 GB, Llama-2-7B, and 200 prompts per rate. The column tl gives the system-wide batch-size cap; the decode-stage ranges specify the segment partition over the 500-token decode horizon. The per-stage thresholds are induced from these two quantities by the continuation-based allocation rule in Section 6.2.

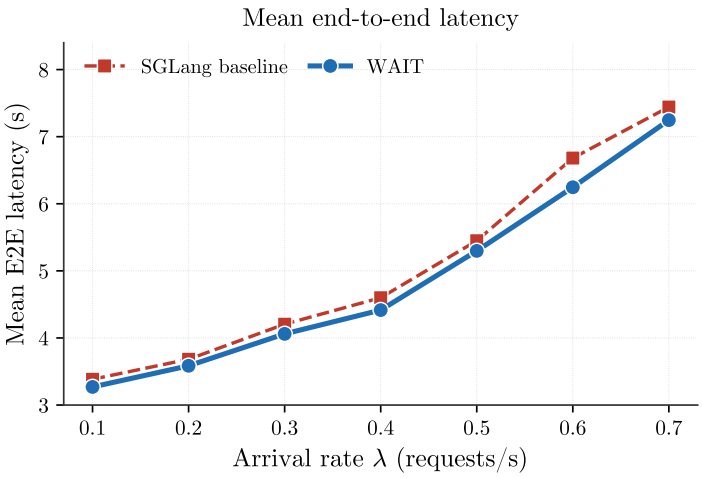

Figure 20 Real-data GPU experiment on SGLang 0.5.7 with the lmsys-chat-1m prompt distribution: mean end-to-end latency versus arrival rate for the SGLang baseline scheduler and WAIT. The WAIT runs use the rate-specific parameter settings in Table 3.

H.3. Repeated-run variability

The main arrival-rate figures in Section 6 report averages over 10 independent simulation replications at each plotted configuration. To examine sampling variability near the stability boundary, we additionally repeat the near-boundary configurations 20 times and report the corresponding deviations.

The repeated-run diagnostic preserves the same latency ordering as the main figures across independent arrival draws.

Workload	λ	vLLM (s)	Sarathi (s)	WAIT (s)
Single-type	22	18.23 ± 2.74	1.85 ± 0.33	**1.03 ± 0.04**
Single-type	23	26.78 ± 3.35	5.08 ± 1.72	**1.29 ± 0.10**
Multi-type W3	22	56.70 ± 2.22	4.06 ± 1.66	**2.36 ± 0.57**
Multi-type W3	23	80.25 ± 2.37	8.41 ± 1.74	**5.66 ± 1.88**

Table 4 Repeated-run reproducibility check at the near-boundary arrival rates. Each entry reports mean end-to-end latency together with the sample standard deviation across repeated simulation runs.

H.4. Prefill-decode-disaggregated deployment

Production LLM serving increasingly uses a *prefill-decode disaggregation* (PD) architecture, in which prefill and decode are executed by separate services and the KV cache is transferred between them (Patel et al. 2024, Zhong et al. 2024). This architecture closely matches the modeling assumptions in Section 2. Because the decode server no longer mixes prefill and decode work,

each iteration processes one token for each active post-prefill job, and the iteration time is driven primarily by active KV-cache usage. The linear iteration-time model in (1) is therefore especially appropriate in this setting. The corresponding control decision is how many post-prefill jobs to keep in concurrent decoding; the WAIT threshold implements this decision as an upper bound on the decode-side service population.

We implement this setting in Vidur by using a decode-only request generator: each request enters the simulated decode server after prefill has completed, with post-prefill context length 630 tokens and decode length 20 (`p630d20`). Arrivals follow a Poisson process with $\lambda \in \{100, 150, \ldots, 500\}$ requests/s. In this decode-only setting, Sarathi's chunked-prefill rule no longer changes the scheduling decision, so vLLM's PD scheduler (`vLLM (PD)`) represents the baseline. Figure 21 reports mean end-to-end decode latency: WAIT improves over vLLM at every rate, with the relative gap ranging from 45% at $\lambda = 100$ to 55% at $\lambda = 500$. Thus, when prefill-decode interactions are removed, the same threshold rule continues to improve decode-side delay.

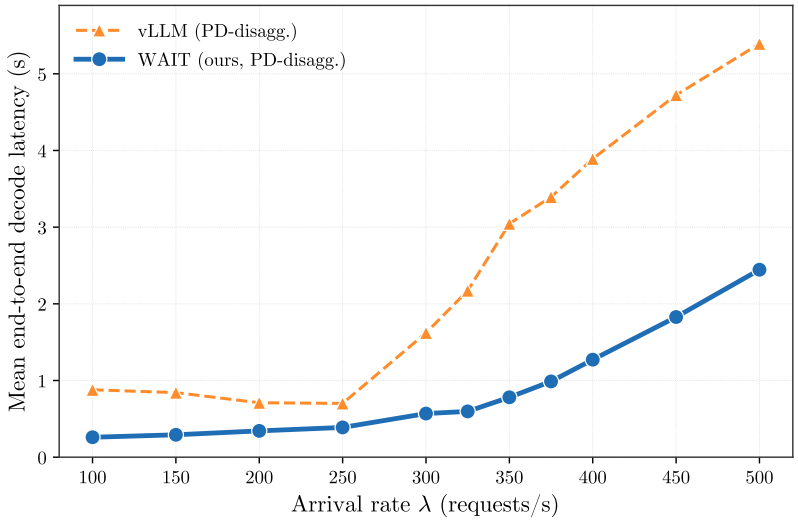

Figure 21 Prefill-decode-disaggregated deployment (`p630d20`, decode-only): mean end-to-end decode latency versus arrival rate. WAIT attains between 45% and 55% latency reduction over the vLLM PD baseline across the tested arrival-rate range.